

UNIVERSITATEA DE STAT DIN MOLDOVA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
DEPARTAMENTUL DE INFORMATICĂ

STRATON ALEXANDRU

PROIECTAREA ȘI DEZVOLTAREA UNUI PORTAL WEB PENTRU
GESTIONAREA INTERNSHIP-URILOR STUDENȚILOR

0613.5 INFORMATICĂ APLICATĂ

Teză de licență

Șef de departament: _____ CAPCELEA Titu, dr., conf. univ.
(semnătura)

Conducătorul științific: _____ DRUMEA Cezar, asis. univ.
(semnătura)

Autorul: _____ STRATON Alexandru, st. gr. IA2302
(semnătura)

CHIȘINĂU – 2026

CUPRINS

LISTA ABREVIERILOR	4
INTRODUCERE	5
I FUNDAMENTE TEORETICE, ANALIZA ȘI FORMULAREA PROBLEMEI DE CERCETARE	7
1.1. Fundamente teoretice ale aplicațiilor web moderne	7
1.2. Autentificarea și autorizarea în sisteme fără stare	9
1.3. Modelul relațional și stratul de abstractizare a persistenței.....	11
1.4. Principii de componentizare gestionarea stării și navigarea în interfața utilizator	12
1.5. Stocarea externalizată a fișierelor în mediul cloud.....	14
1.6. Analiza domeniului și contextul problemei	15
1.7. Formularea problemei de cercetare	16
1.8. Arborele problemei aplicat.....	17
1.9. Obiectivele cercetării	18
1.10. Concluzii la Capitolul I.....	18
II STUDII DE CAZ ȘI SOLUȚIILE PROPUSE PENTRU INTERNSHIPFINDER	19
2.1. Scopul capitolului și metodologia cercetării	19
2.2. Analiza portalurilor de recrutare din Republica Moldova	19
2.2.1. Rabota.md platformă generalistă de recrutare cu acoperire largă a pieței muncii	20
2.2.2. Jobber.md platformă modernă orientată spre experiența utilizatorului	20
2.3. Stiva tehnologică a InternshipFinder evaluare comparativă și decizii arhitecturale.....	21
2.3.1. Arhitectura generală monolit modular cu separare client server	21
2.3.2. Interfața utilizatorului, bibliotecă de randare, stilizare și rutare.....	22
2.3.3. Biblioteci utilitare pentru funcționalități specifice	22
2.3.4. Componenta de server, cadru de aplicație, securitate și persistență	23
2.3.5. Stocarea fișierelor în cloud.....	24
2.3.6. Infrastructura de dezvoltare și producție.....	25
2.3.7. Internaționalizarea interfeței	25
2.4. Arborele soluțiilor.....	26
2.5. Concluzii la Capitolul II.....	27
III PROIECTAREA DETALIATĂ ȘI IMPLEMENTAREA SOLUȚIEI INTERNSHIPFINDER	28
3.1. Scopul capitolului și cadrul de implementare	28
3.2. Structura soluției implementate, client server și servicii auxiliare	28

3.3.	Configurarea aplicației și dependențele principale.....	29
3.4.	Implementarea securității stateless, JWT și Spring Security	30
3.5.	Implementarea fluxurilor de autentificare, reîmprospătare și parolă unică.....	32
3.6.	Persistența PostgreSQL și standardizarea mediului de dezvoltare	34
3.7.	Implementarea stocării fișierelor în Azure Blob Storage.....	34
3.8.	Implementarea interfeței de programare a aplicației pe roluri	35
3.9.	Implementarea interfeței utilizatorului, integrare, sesiune și rutare protejată	36
3.10.	Implementarea funcționalității CV Builder.....	37
3.11.	Internaționalizarea și persistarea preferinței de limbă	38
3.12.	Concluzii la Capitolul III	39
CONCLUZII		40
BIBLIOGRAFIE		42
ANEXE.....		44
Anexa 1. Tabelele problemei de cercetare		44
Anexa 2. Tabelele soluțiilor propuse		45
Anexa 3. Configurația Maven și dependențele proiectului.....		46
Anexa 4. Configurația externalizată a aplicației.....		47
Anexa 5. Configurația securității Spring Security.....		48
Anexa 6. Componentele JWT, generare și filtrare		49
Anexa 7. Controlerul de autentificare și serviciul de upload		50
Anexa 8. Componentele frontend pentru protecția rutelor.....		51
Anexa 9. Modelul pentru token-uri OTP și transmiterea prin email		52
Anexa 10. Controlere pentru API pe roluri.....		53
Anexa 11. Configurația interfeței utilizatorului, Vite, Axios și rutare		54
Anexa 12. Funcționalitatea CV Builder.....		56
DECLARAȚIE PRIVIND ASUMAREA RĂSPUNDERII.....		58
DECLARAȚIE PRIVIND CONTRIBUȚIA PERSONALĂ		59

LISTA ABREVIERILOR

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CORS	Cross-Site Request Forgery
CV	Curriculum Vitae
DTO	Data Transfer Object
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / HyperText Transfer Protocol Secure
JPA	Java Persistence API
JVM	Java Virtual Machine
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
OTP	One-Time Password
PDF	Portable Document Format
REST	Representational State Transfer
SDK	Software Development Kit
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface

INTRODUCERE

Societatea contemporană este marcată de o digitalizare accelerată a tuturor proceselor sociale și economice, iar piața muncii nu face excepție de la această tendință generală. Dacă în urmă cu câteva decenii căutarea unui loc de muncă presupunea în principal consultarea ziarelor tipărite, depunerea dosarelor fizice și parcurgerea unor procese lente și costisitoare, astăzi așteptările sunt fundamental diferite. Atât candidații, cât și angajatorii au ajuns să considere viteza, transparența și accesibilitatea platformelor digitale nu ca pe un avantaj competitiv, ci ca pe o cerință minimă a oricărui canal de recrutare modern. În acest context, absența unor soluții dedicate pentru segmente specifice ale pieței muncii - cum este cel al stagiilor de practică pentru studenți - devine cu atât mai vizibilă și mai costisitoare.

Stagiile de practică, denumite curent internship-uri, ocupă un loc aparte în traiectoria profesională a unui student. Ele reprezintă primul contact semnificativ cu mediul profesional real, ocazia de a transforma cunoștințele teoretice în competențe aplicabile și, nu de puține ori, punctul de plecare al unei cariere. Paradoxal, tocmai această etapă crucială este adesea gestionată prin mijloace improvizate: grupuri pe rețele sociale, mesaje directe, foi de calcul distribuite prin e-mail sau anunțuri afișate pe avizierele facultăților. Nivelul de digitalizare al procesului este, în cele mai multe cazuri, superficial - există o prezență online, dar nu un sistem coerent care să susțină întregul flux, de la publicarea ofertei și candidatura studentului, până la urmărirea statusului aplicației și generarea documentelor necesare.

Lucrarea de față pornește tocmai de la această realitate și propune o soluție concretă: InternshipFinder, un portal web full-stack destinat gestionării integrate a internship-urilor studenților. Platforma pune față în față trei categorii de actori - studenți, recrutori și administratori de sistem -, fiecare cu propriile nevoi, drepturi și responsabilități în cadrul aplicației. Studentul poate să-și completeze profilul, să încarce CV-ul, să exploreze ofertele disponibile și să urmărească statusul candidaturilor depuse. Recrutorul poate publica oferte de internship, consulta profilurile candidaților și gestiona aplicațiile primite. Administratorul supervisează întreg ecosistemul: gestionează utilizatorii, monitorizează activitatea platformei și menține integritatea datelor.

Construirea unui astfel de sistem nu este, însă, un demers trivial. Ea presupune soluționarea simultană a mai multor probleme tehnice interdependente: cum se asigură că utilizatorul autentificat nu poate accesa resursele altui utilizator, cum se garantează că datele rămân consistente în condiții de acces concurent, cum se gestionează eficient fișierele binare fără a supraîncărca baza de date, cum se

construiește o interfață reactivă care să funcționeze fluent pe orice dispozitiv, și cum se face toate acestea într-un mod suficient de general încât sistemul să poată fi extins fără a fi rescris de la zero. Răspunsurile la aceste întrebări formează coloana vertebrală a lucrării de față.

Structura lucrării urmează o logică ascendentă, de la abstract la concret. Primul capitol stabilește cadrul teoretic, prezentând principiile fundamentale care justifică fiecare decizie arhitecturală: natura comunicării web, mecanismele de securitate, modelul relațional de stocare a datelor, paradigma componentizării interfeței și principiile stocării în cloud. Tot în primul capitol sunt formulate problema de cercetare, arborele problemei și obiectivele care ghidează întreaga lucrare. Al doilea capitol prezintă analiza platformelor existente din piața locală și justificarea comparativă a stivei tehnologice alese, construind arborele soluțiilor ca răspuns direct la cauzele identificate. Al treilea capitol detaliază implementarea propriu-zisă, urmărind fiecare componentă a sistemului de la configurare până la funcționarea în producție.

Importanța lucrării depășește contextul strict academic. InternshipFinder este conceput ca o soluție cu aplicabilitate reală, capabilă să servească drept fundament pentru un portal funcțional la nivelul unei instituții de învățământ superior sau al unei organizații de mediere a stagiilor. Prin această lucrare se urmărește demonstrarea faptului că o aplicație web modernă, securizată și extensibilă poate fi construită cu resurse limitate, pornind de la o înțelegere riguroasă a problemei și de la decizii arhitecturale bine fundamentate.

I FUNDAMENTE TEORETICE, ANALIZA ȘI FORMULAREA PROBLEMEI DE CERCETARE

Soliditatea oricărui sistem informatic derivă, în ultimă instanță, din calitatea fundamentelor conceptuale pe care a fost construit. Un sistem care a fost gândit superficial, pornind direct de la instrumente și tehnologii fără o înțelegere profundă a principiilor care le guvernează, este sortit să acumuleze datorii tehnice greu de rambursat și să devină fragil în fața cerințelor de extindere. Tocmai de aceea, înainte de a trece la orice decizie concretă de proiectare sau implementare, este esențial să fie clarificate principiile teoretice care justifică structura aleasă și care conferă coerență întregului demers. Capitolul de față urmărește acest obiectiv: să construiască cadrul conceptual în care se înscrie portalul InternshipFinder, analizând cu rigurozitate fiecare domeniu teoretic relevant, de la natura comunicării web și a securității, până la persistența datelor, stocarea externalizată a fișierelor și construirea interfeței utilizatorului.

1.1. Fundamente teoretice ale aplicațiilor web moderne

Evoluția aplicațiilor web de-a lungul ultimelor trei decenii a fost marcată de o tensiune permanentă între simplitate și complexitate. Primele aplicații web erau, prin natura lor, simple: un server genera dinamic conținut HTML pe care browserul îl afișa, iar interacțiunea utilizatorului declanșa reîncărcarea completă a paginii. Această arhitectură, deși funcțională pentru cazurile de utilizare simple, a devenit rapid inadecvată pe măsură ce așteptările utilizatorilor au crescut și aplicațiile au trebuit să gestioneze stări complexe, colaborare în timp real și experiențe de utilizare competitive cu aplicațiile desktop.

Răspunsul industriei la această provocare a fost separarea preocupărilor la nivel arhitectural: componenta de interfață și componenta de logică a domeniului au fost despărțite în entități distincte, comunicând printr-un protocol bine definit. Această separare, departe de a fi o simplă convenție de stil, reflectă o realitate practică profundă: echipele de dezvoltare pot lucra în paralel pe cele două componente, fiecare putând fi scalată independent în funcție de cerințe, iar înlocuirea uneia dintre ele nu necesită rescrierea celeilalte. Dintr-o perspectivă de inginerie software, aceasta reprezintă aplicarea directă a principiului de separare a responsabilităților la nivel de sistem. [1]

La baza comunicării dintre client și server se află protocolul HTTP, definit prin standardul RFC 9110, care stabilește semantica operațiilor, formatul mesajelor și semnificația codurilor de stare. Protocolul este, prin proiectare, lipsit de stare: fiecare pereche cerere-răspuns este un schimb

independent, serverul neputând presupune că o cerere este legată de una anterioară decât dacă aceasta aduce în mod explicit informațiile necesare. Această proprietate fundamentală nu reprezintă o limitare accidentală, ci o alegere deliberată de proiectare, care conferă sistemului o scalabilitate naturală: orice server din sistem poate procesa orice cerere, indiferent de istoricul interacțiunilor cu acel client. [1]

Caracterul stateless al protocolului are însă consecințe profunde asupra modului în care sunt proiectate autentificarea, autorizarea și managementul stării aplicației. Întrucât serverul nu reține contextul utilizatorului între cereri, acesta trebuie re-stabilit la fiecare schimb, fie prin mecanisme de sesiune server-side, fie prin tokenuri transmise de client odată cu fiecare cerere. Alegerea dintre aceste două abordări nu este neutră: ea influențează scalabilitatea sistemului, complexitatea implementării și nivelul de securitate atins. Un sistem bazat pe sesiuni server-side este mai simplu de implementat inițial, dar devine o sursă de complexitate operațională pe măsură ce numărul de instanțe crește. Un sistem bazat pe tokenuri este mai complex de configurat corect, dar scalează natural și elimină necesitatea sincronizării stării între instanțe.

Un principiu larg adoptat în proiectarea interfeței dintre client și server este orientarea pe resurse: aplicația expune entități ale domeniului și operații asupra lor, fiecare resursă identificată printr-un locator unic și accesibilă prin operații cu semantică prestabilită. HTTP definește această semantică prin metodele sale: operația de obținere este separată de cea de creare, care este separată de cea de actualizare, care la rândul ei este separată de cea de ștergere. Această claritate semantică nu este un lux estetic, ci o necesitate practică: ea permite clienților de orice natură - browsere, aplicații mobile, servicii terțe - să interacționeze cu serverul fără cunoștințe prealabile despre implementarea sa internă. [1]

Contractul stabilit între client și server printr-un API bine definit este, în esență, documentul care face posibilă dezvoltarea paralelă și independentă a celor două componente. Un contract stabil specifică forma cererilor acceptate, datele necesare, formatul răspunsurilor așteptate și codurile de eroare posibile pentru fiecare situație. Într-un portal cu mai multe roluri și fluxuri de lucru complexe, un astfel de contract devine cu atât mai important: fiecare rol interacționează cu seturi diferite de resurse, iar controlul accesului trebuie corelat precis cu contractul. O eroare în această corelație nu produce un bug minor, ci o vulnerabilitate de securitate cu consecințe potențial severe.

Codurile de stare HTTP joacă un rol esențial în comunicarea semanticii operației: succesul este comunicat prin coduri din familia 2xx, cererile invalide prin coduri 4xx, iar erorile interne prin coduri 5xx. Respectarea acestei semantici nu este doar o chestiune de bune practici, ci o necesitate funcțională: clienții moderni - inclusiv bibliotecile de comunicare HTTP utilizate în frontend - iau decizii bazate pe

codul de stare primit, de la afișarea unui mesaj de eroare potrivit până la declanșarea unui mecanism de reînnoire a tokenului de autentificare.

1.2. Autentificarea și autorizarea în sisteme fără stare

Securitatea unui sistem informatic reprezintă, fără îndoială, una dintre preocupările cu cel mai mare impact practic asupra utilizatorilor finali. Un sistem nesecurizat nu este doar vulnerabil tehnic - el subminează încrederea utilizatorilor, poate expune date sensibile și poate genera consecințe juridice și reputaționale grave pentru organizația care îl operează. Înțelegerea profundă a principiilor care stau la baza securității aplicațiilor web moderne este, prin urmare, o condiție necesară, nu opțională, pentru proiectarea oricărui sistem care gestionează date personale sau acces diferențiat la resurse.

Distincția conceptuală dintre autentificare și autorizare este fundamentală și merită subliniată cu claritate. Autentificarea este procesul prin care un sistem verifică identitatea unui utilizator: răspunde la întrebarea „*cine ești?*” și stabilește, pe baza unor dovezi furnizate de utilizator, că acesta este cel pretinde a fi. Autorizarea este procesul care urmează autentificării și determină ce are utilizatorul identificat voie să facă: răspunde la întrebarea „*ce îți este permis?*” și restricționează accesul la resurse sau operații pe baza drepturilor asociate identității verificate. Cele două procese sunt distincte, dar inseparabile în practică: autorizarea fără autentificare este lipsită de sens, iar autentificarea fără autorizare este incompletă.

Într-un sistem lipsit de stare, autentificarea trebuie să permită serverului să recunoască identitatea utilizatorului la fiecare cerere, fără a consulta un registru de sesiuni active. Soluția conceptuală pentru această provocare este transmiterea dovezii de identitate odată cu fiecare cerere, sub forma unui token verificabil. Ideea fundamentală este că serverul emite, la autentificarea cu succes, un artefact criptografic care „*codifică*” identitatea utilizatorului și drepturile sale, artefactul pe care clientul îl prezintă la fiecare cerere ulterioară. Serverul poate verifica autenticitatea acestui artefact exclusiv pe baza cheii criptografice pe care o deține, fără a accesa nicio bază de date de sesiuni. Aceasta este esența autentificării stateless. [2]

Standardul RFC 7519 definește formatul JSON Web Token ca mecanism compact și verificabil pentru reprezentarea informațiilor transferate între două entități. Structura unui token de acest tip este compusă din trei secțiuni codificate: antetul, care specifică algoritmul de semnare utilizat; corpul, care conține afirmațiile despre utilizator și metadatele tokenului; și semnătura, care garantează că niciuna dintre primele două secțiuni nu a fost alterată după emitere. Mecanismul de semnare utilizat - în mod

tipic unul asimetric sau unul bazat pe chei secrete partajate - face imposibilă falsificarea unui token valid fără cunoașterea cheii secrete a serverului. [2]

Printre metadatele esențiale ale unui token se numără momentul de expirare, care limitează fereastra de utilizare a tokenului și reduce impactul în cazul compromiterii acestuia. Un token furat care a expirat nu mai poate fi utilizat de un atacator, ceea ce face din gestionarea corectă a duratei de viață un element critic al securității sistemului. Practica curentă separă tokenurile în două categorii cu durate diferite: tokenuri de acces, cu durată scurtă de viață (tipic câteva minute până la câteva ore), utilizate pentru autentificarea cererilor obișnuite; și tokenuri de reînnoire, cu durată mai lungă, utilizate exclusiv pentru obținerea unor noi tokenuri de acces fără a solicita re-autentificarea utilizatorului.

Validarea unui token la primirea unei cereri implică mai mulți pași, fiecare critic: verificarea semnăturii criptografice, care garantează autenticitatea tokenului; verificarea termenului de expirare, care garantează că tokenul nu a depășit durata sa de viață; și, opțional, verificarea emitentului și a audienței declarate, care garantează că tokenul a fost emis de serverul corect pentru aplicația corectă. Orice token care eșuează oricare dintre aceste verificări trebuie respins, iar cererea asociată tratată ca neautorizată și returnată cu codul de stare corespunzător. [2]

Controlul accesului pe roluri adaugă un strat suplimentar de complexitate controlată: nu orice utilizator autentificat are voie să efectueze orice operație. Diferitele categorii de utilizatori - studenți, recrutori, administratori în cazul portalului InternshipFinder - au drepturi și responsabilități diferite, care trebuie reflectate coerent atât în logica de backend, cât și în experiența de interfață. Implementarea corectă a acestui control presupune că drepturile de acces sunt verificate în stratul de logică al aplicației, independent de ceea ce afișează sau ascunde interfața utilizatorului: ascunderea unui buton în interfață nu este o măsură de securitate, ci doar o îmbunătățire a experienței utilizatorului.

Un aspect adesea subestimat în proiectarea securității este comportamentul corect al sistemului în condițiile expirării tokenului în timpul unei sesiuni active de utilizare. Un utilizator care lucrează activ pe platformă timp de mai multe ore nu trebuie să fie forțat să se re-autentifice manual la fiecare expirare a tokenului de acces. Mecanismul de reînnoire automată - prin care clientul detectează expirarea tokenului, solicită transparent un nou token de acces pe baza tokenului de reînnoire și reia cererea inițială - este esențial pentru o experiență de utilizare coerentă, fără a compromite securitatea sistemului.

1.3. Modelul relațional și stratul de abstractizare a persistenței

Persistența datelor reprezintă una dintre preocupările fundamentale ale oricărei aplicații care trebuie să supraviețuiască dincolo de o singură sesiune de utilizare. Alegerea modelului de stocare nu este o decizie tehnică minoră - ea influențează profund structura întregii aplicații, modul în care datele sunt organizate, relaționate și interogate, și capacitatea sistemului de a menține consistența în condiții de acces concurent.

Modelul relațional, introdus de Edgar Codd în 1970 și materializat în sisteme de gestiune a bazelor de date utilizate pe scară industrială, organizează datele sub formă de relații - denumite curent tabele -, compuse din attribute - denumite coloane - și tupluri - denumite rânduri. Forța modelului relațional nu rezidă în simplitatea acestei structuri, ci în mecanismele formale care garantează integritatea datelor: cheile primare identifică unic fiecare înregistrare, cheile externe exprimă relații de referință între tabele, constrângerile de unicitate și de valoare non-nulă previn introducerea datelor invalide. [3]

Proprietățile ACID - atomicitate, consistență, izolare și durabilitate - definesc garanțiile pe care un sistem de gestiune relațional le oferă tranzacțiilor. Atomicitatea garantează că o tranzacție este fie executată în întregime, fie nu este executată deloc, eliminând stările parțiale. Consistența garantează că tranzacția duce baza de date dintr-o stare validă în altă stare validă, respectând toate constrângerile definite. Izolarea garantează că execuția concurentă a mai multor tranzacții produce același rezultat ca și execuția lor secvențială. Durabilitatea garantează că efectele unei tranzacții confirmate sunt persistate chiar și în cazul unui eșec al sistemului. [3].

Aceste proprietăți sunt esențiale într-un portal în care datele sunt profund interdependente și în care acțiunile mai multor utilizatori pot interfera. Imaginați-vă scenariul în care un student depune o candidatură exact în momentul în care un recrutor șterge oferta corespunzătoare. Fără garanțiile ACID, sistemul ar putea ajunge într-o stare inconsistentă - o candidatură asociată unei oferte inexistente. Cu garanțiile ACID, una dintre cele două operații va eșua în mod controlat, iar baza de date va rămâne într-o stare validă.

Normalizarea datelor reprezintă un alt principiu fundamental al modelului relațional. Ea urmărește organizarea datelor în tabele distincte, astfel încât fiecare fapt să fie stocat într-un singur loc, eliminând redundanța și dependențele transitive. O schemă normalizată corespunzător reduce riscul anomaliilor de actualizare - situații în care modificarea unui fapt într-un singur loc lasă copii ale acestuia în alte locuri neactualizate -, simplificând mentenanța și crescând integritatea datelor. [3]

Aplicațiile moderne sunt construite în paradigma orientată pe obiecte, în care datele sunt modelate ca entități cu identitate, stare și comportament, interconectate prin relații. Discrepanța structurală fundamentală dintre reprezentarea tabulară a bazei de date și modelul obiectual al aplicației - denumită în literatură „*impedance mismatch*” - este o sursă clasică de complexitate care apare la granița dintre stratul de logică și stratul de persistență. Dacă această graniță este gestionată manual, prin scrierea de interogări SQL brute în logica aplicației, rezultatul este cod fragil, dificil de refactorizat și predispus la erori.

Stratul de abstractizare a persistenței, cunoscut sub acronimul ORM (Object-Relational Mapping), rezolvă această discrepanță prin maparea automată dintre entitățile obiectuale ale aplicației și tabelele bazei de date. Dezvoltatorul lucrează exclusiv cu obiecte și colecții de obiecte la nivel de cod, iar traducerea în operații de bază de date - interogări, inserții, actualizări, ștergeri - rămâne responsabilitatea infrastructurii de persistență. Avantajele sunt multiple: codul de aplicație devine mai expresiv și mai ușor de înțeles, schimbările de schemă pot fi propagate mai simplu, iar portabilitatea între furnizori de baze de date este îmbunătățită.

Un avantaj semnificativ al utilizării unui strat de persistență bine structurat este că încurajează o arhitectură pe straturi clare și bine delimitate. Entitățile descriu structura datelor, componenta de acces la date definește operațiile de interogare specifice domeniului, stratul de serviciu conține logica de business și gestionează tranzacțiile, iar stratul de control expune operațiile disponibile prin interfața externă. Această separare nu este doar estetică - ea face ca fiecare componentă să fie testabilă izolat, ușor de înlocuit și de extins fără a afecta celelalte straturi.

1.4. Principii de componentizare gestionarea stării și navigarea în interfața utilizator

Interfața utilizatorului reprezintă punctul de contact direct dintre sistem și persoana care îl utilizează. Oricât de solid ar fi un sistem din perspectiva securității, persistenței sau performanței, valoarea sa practică este mediată de calitatea interfeței prin care utilizatorul îl accesează. O interfață slabă poate face un sistem bun să pară nefolositor, în timp ce o interfață bine gândită poate transforma un sistem modest într-un instrument apreciat. Această realitate nu este nouă, dar a dobândit o importanță crescută odată cu democratizarea accesului la internet și cu creșterea așteptărilor utilizatorilor obișnuiți cu aplicații mobile și web de înaltă calitate.

Paradigma componentizării interfeței a apărut ca răspuns la complexitatea crescândă a aplicațiilor web. Abordarea tradițională - în care interfața era construită ca un bloc monolitic de cod HTML, CSS și

JavaScript strâns cuplate - funcționa acceptabil pentru aplicații simple, dar devenea rapid inabordabilă pe măsură ce complexitatea creștea. Componentizarea propune o soluție elegantă: aplicația este construită din unități independente, fiecare cu propria stare, logică de randare și interfață publică, care pot fi compuse și reutilizate în diferite contexte. [5]

Un component bine proiectat este izolat: nu depinde de starea globală a aplicației, nu produce efecte secundare neașteptate și poate fi testat independent de restul sistemului. Această izolare nu este doar o proprietate dezirabilă din perspectiva calității codului - ea are consecințe directe asupra vitezei de dezvoltare, ușurinței de debugging și capacității de a introduce noi funcționalități fără a perturba funcționalitățile existente. [5]

Aplicațiile de tip SPA (Single Page Application) au revoluționat experiența de utilizare a aplicațiilor web prin eliminarea reîncărcărilor complete ale paginii la fiecare navigare. Într-o aplicație SPA, la prima accesare este descărcat întregul cod al aplicației, iar navigările ulterioare sunt gestionate exclusiv de codul JavaScript din browser, fără a solicita serverului pagini HTML noi. Serverul este consultat doar pentru datele necesare, returnate de obicei în format JSON. Rezultatul perceput de utilizator este o experiență fluidă, comparabilă cu cea a unei aplicații desktop, în care tranzițiile între secțiuni sunt instantanee și starea aplicației este păstrată pe parcursul navigării.

Mecanismul de actualizare selectivă a interfeței, utilizat de bibliotecile moderne de construire a interfețelor, reprezintă o inovație cu impact semnificativ asupra performanței percepute. Ideea fundamentală este că, în loc să refacă întreaga structură vizuală a paginii la fiecare modificare de stare, sistemul calculează diferența minimă dintre starea curentă și cea dorită și aplică exact modificările necesare structurii vizuale a browserului. Această abordare reduce semnificativ costul operațiilor de actualizare a interfeței, permițând aplicațiilor să fie reactive și fluente chiar și atunci când gestionează seturi mari de date sau actualizări frecvente.

Gestionarea stării reprezintă una dintre provocările cele mai complexe ale construirii aplicațiilor de interfață moderne. Starea unei aplicații include tot ceea ce trebuie să fie reținut pentru ca interfața să afișeze informațiile corecte: identitatea utilizatorului autentificat, conținutul formularelor completate, datele încărcate de la server, preferințele de afișare și altele. Această stare trebuie să fie accesibilă în locurile potrivite ale aplicației, actualizată sincron atunci când se schimbă și persistă acolo unde este necesar dincolo de o singură sesiune.

Într-o aplicație cu autentificare și roluri multiple, starea de autentificare este un caz particular de stare globală: identitatea utilizatorului curent, rolul său și drepturile asociate trebuie să fie accesibile din

orice punct al aplicației care ia decizii de afișare sau de navigare. Transmiterea acestei stări manual prin întregul arbore de componente - practică denumită „*prop drilling*” - este inefficientă și fragilă. Mecanismele de context ale bibliotecilor moderne de interfață oferă o alternativă elegantă: starea de autentificare este definită într-un singur loc și pusă la dispoziția oricărei componente care o solicită, fără a perturba structura arborescentă a componentelor.

Navigarea protejată reprezintă o altă dimensiune importantă a interfeței unui portal cu roluri multiple. Rutele aplicației nu sunt egale ca acces: unele sunt publice, accesibile oricui; altele sunt protejate, accesibile doar utilizatorilor autentificați; altele sunt restricționate pe rol, accesibile doar utilizatorilor cu rolul potrivit. Implementarea corectă a navigării protejate implică verificarea stării de autentificare și a rolului la fiecare tranziție de rută, redirecționarea automată a utilizatorilor care încearcă să acceseze rute nepermise și reținerea rutei solicitate pentru redirecționare ulterioară, după autentificarea cu succes.

Internaționalizarea reprezintă proiectarea aplicației astfel încât adăugarea suportului pentru o nouă limbă să fie o operație de configurare, nu de modificare a codului. Aceasta implică externalizarea tuturor textelor afișate în fișiere de resurse dedicate, organizate pe limbă, și înlocuirea textelor hardcodate din cod cu referințe la cheile din aceste fișiere. Localizarea efectivă - traducerea conținutului - devine astfel separată de codul aplicației, putând fi realizată de traducători fără cunoștințe de programare. Preferința lingvistică a utilizatorului trebuie persistată astfel încât să fie restaurată la fiecare vizită, eliminând necesitatea re-selectării limbii.

1.5. Stocarea externalizată a fișierelor în mediul cloud

Gestionarea fișierelor generate de utilizatori ridică o problemă arhitecturală distinctă față de stocarea datelor structurate. Fișierele - documente de tip CV, fotografii de profil, rapoarte generate - sunt prin natura lor obiecte binare de dimensiuni variabile, cu caracteristici de acces fundamental diferite față de datele structurate: sunt citite rar și în întregime, nu sunt niciodată parțial actualizate, nu sunt interogate după conținut și pot ajunge la dimensiuni care fac inefficientă stocarea lor în coloanele unei baze de date relaționale.

Motivele pentru care stocarea fișierelor direct în baza de date relațională este o abordare problematică sunt multiple și bine documentate. Bazele de date relaționale sunt optimizate pentru operații de interogare pe date structurate, nu pentru transferul unor volume mari de date binare. Stocarea fișierelor în coloane de tip BLOB (Binary Large Object) ocupă spațiu în memoria de lucru a motorului de baze de

date, afectând performanța operațiilor de interogare. Backup-urile bazei de date devin considerabil mai mari și mai lente. Scalarea componentei de stocare este cuplată forțat cu scalarea componentei de persistență relațională, chiar dacă nevoile celor două sunt complet diferite.

Serviciile de stocare a obiectelor în cloud reprezintă soluția arhitecturală consacrată pentru această problemă. Ele sunt concepute special pentru stocarea eficientă a fișierelor de orice dimensiune, oferind disponibilitate ridicată, redundanță geografică automată, politici de acces granulare și scalabilitate practic nelimitată. [7] Integrarea unui astfel de serviciu în arhitectura unui portal separă în mod clar stocarea fișierelor de persistența datelor structurate, permițând fiecărei componente să fie scalată independent, conform propriilor nevoi.

Din perspectiva integrării în arhitectura aplicației, serviciile de stocare în cloud expun interfețe programatice prin care componenta de server poate încărca fișiere, genera adrese de acces și controla drepturile de vizualizare. O practică importantă în gestionarea accesului la fișierele stocate este generarea de adrese de acces temporare, valabile pentru o perioadă limitată și pentru un scop specific. Această abordare permite ca fișierele private ale unui utilizator să fie accesate direct de browser, fără a trece prin serverul aplicației, dar fără a fi expuse public pe durată nedeterminată. [7]

Stocarea în baza de date a referinței către fișierul cloud - nu a conținutului fișierului - este un alt element important al acestei arhitecturi. Referința poate fi un identificator unic sau o adresă de localizare, care poate fi utilizată ulterior pentru a genera adrese de acces temporare sau pentru a șterge fișierul din serviciul cloud. Această abordare menține schemă relațională simplă și interogabilă, în timp ce conținutul binar este gestionat de un serviciu specializat.

1.6. Analiza domeniului și contextul problemei

InternshipFinder este o aplicație web de tip portal, concepută pentru a susține fluxuri de interacțiune structurate între trei categorii de actori: studenți, recrutori și administratori de sistem. Specificul unui portal multi-rol implică o serie de cerințe simultane care se influențează reciproc și care trebuie să fie abordate în mod integrat pentru a rezulta un sistem coerent.

Studentul, ca principal beneficiar al platformei, are nevoie să-și poată construi și menține un profil profesional complet, să încarce documentele de candidatură, să exploreze ofertele disponibile, să aplice pentru internship-urile care îi corespund și să urmărească statusul candidaturilor depuse. Fiecare dintre aceste operații implică accesul la resurse private - profilul propriu, documentele proprii, candidaturile proprii - și trebuie să fie protejat față de accesul altor utilizatori, inclusiv al altor studenți.

Recrutorul are nevoi diferite: publicarea și gestionarea ofertelor de internship ale organizației pe care o reprezintă, consultarea candidaturilor primite, accesul la profilurile studenților care au aplicat și actualizarea statusului aplicațiilor. Recrutorul nu trebuie să aibă acces la profilurile studenților care nu au aplicat la ofertele sale, nici la ofertele altor recrutori, nici la funcționalitățile administrative ale platformei.

Administratorul gestionează întregul ecosistem al platformei: verifică și moderează utilizatorii, poate bloca sau activa conturi, supraveghează ofertele publicate și menține integritatea datelor. Acțiunile administrative au un impact sistemic și trebuie să fie înregistrate pentru auditabilitate: cine a efectuat ce acțiune, când și din ce locație.

Analiza domeniului evidențiază că problema centrală nu constă în absența tehnologiilor disponibile pentru fiecare componentă în parte, ci în dificultatea integrării lor coerente într-un sistem unificat. Fiecare componentă - autentificarea, autorizarea, persistența, stocarea fișierelor, interfața, internaționalizarea - poate fi implementată separat cu soluții cunoscute și bine documentate. Valoarea reală și provocarea arhitecturală apar din coerența ansamblului: asigurarea că securitatea nu lasă fisuri la intersecția componentelor, că starea este gestionată consistent, că erorile sunt tratate uniform și că sistemul ca întreg este mai mult decât suma părților sale.

1.7. Formularea problemei de cercetare

Pornind de la fundamentele teoretice prezentate în secțiunile anterioare și de la analiza domeniului de aplicație, problema de cercetare a acestei lucrări poate fi formulată în mod riguros și complet. Formularea problemei nu urmărește să fie exhaustivă în lista tehnologiilor implicate, ci să captureze esența provocării arhitecturale și să identifice tensiunile conceptuale care trebuie rezolvate pentru a obține un sistem funcțional și coerent.

“ Cum poate fi proiectată și implementată o aplicație web full-stack de tip portal - destinată gestionării internship-urilor studenților - care să ofere un flux coerent de autentificare și control al accesului pe roluri în condiții stateless, să asigure securizarea resurselor prin tokenuri verificabile cu durată limitată de viață, să mențină persistența consistentă a datelor structurate într-un model relațional cu ajutorul unui strat de abstractizare a persistenței, să gestioneze eficient stocarea externalizată a fișierelor utilizatorilor printr-un serviciu cloud dedicat, să ofere o interfață modulară bazată pe componente cu navigare protejată și gestionare globală a stării de autentificare, și să suporte internaționalizarea interfeței pentru mai multe limbi - toate acestea integrându-se coerent într-un sistem extensibil, fără contradicții arhitecturale sau puncte de rupere? [1], [2], [3], [5], [7], [8] “

Această formulare capturează complexitatea reală a problemei prin caracterul ei integrator: fiecare condiție enumerată nu poate fi satisfăcută izolat, ci trebuie abordată în conjuncție cu celelalte. Soluția nu poate fi găsită prin identificarea celei mai bune tehnologii pentru fiecare componentă în parte, ci prin proiectarea coerentă a modului în care componentele interacționează, se susțin și se completează reciproc.

1.8. Arborele problemei aplicat

Arborele problemei este un instrument analitic utilizat în analiza sistemelor complexe pentru a structura relația cauzală dintre factorii care contribuie la apariția unei probleme și efectele pe care aceasta le produce. Aplicat în contextul acestei lucrări, el relevă că dificultatea construirii unui portal web securizat și extensibil nu are o cauză unică, ci rezultă dintr-o constelație de constrângeri independente, fiecare cu o origine și un mecanism propriu de propagare.

Identificarea cauzelor nu se face arbitrar, ci prin analiza fiecărei componente a sistemului și a constrângerilor pe care aceasta le impune proiectării. Astfel, natura stateless a protocolului de comunicare impune proiectarea explicită a mecanismelor de autentificare - fără aceasta, serverul nu poate recunoaște utilizatorul. Necesitatea diferențierii accesului pe roluri impune implementarea unui mecanism de autorizare - fără aceasta, orice utilizator autentificat ar putea accesa orice resursă. Complexitatea datelor interdependente impune utilizarea unui model relațional cu constrângeri de integritate - fără aceasta, consistența datelor nu poate fi garantată. Și așa mai departe pentru fiecare cauză identificată. A se vedea Tabelul A1.1 din Anexa 1.

Efectele nerezolvării acestei probleme sunt la fel de importante ca și cauzele. Un sistem cu autentificare defectuoasă poate expune datele personale ale utilizatorilor. Un sistem fără autorizare granulară permite accesul neautorizat la resurse private. O schemă relațională prost proiectată generează inconsistențe de date care pot eroda treptat integritatea întregii platforme. O interfață care nu gestionează corect starea poate produce experiențe confuze sau erori subtile, greu de reprodus și de corectat. Costul de mentenanță al unui sistem care nu a fost proiectat pentru extensibilitate crește exponențial cu complexitatea, până în punctul în care adăugarea de noi funcționalități devine mai costisitoare decât rescrierea sistemului.

1.9. Obiectivele cercetării

Obiectivele cercetării sunt derivate direct din cauzele identificate în arborele problemei și din formularea problemei de cercetare. Fiecare obiectiv răspunde uneia sau mai multor cauze, prefigurând o decizie arhitecturală sau o componentă tehnică a soluției. Definirea clară a obiectivelor servește un scop dublu: pe de o parte, ghidează procesul de proiectare și implementare, asigurând că fiecare decizie poate fi conectată la o nevoie identificată; pe de altă parte, furnizează criteriile față de care soluția implementată poate fi evaluată la finalul lucrării. A se vedea Tabelul A1.2 din Anexa 1.

1.10. Concluzii la Capitolul I

Capitolul I a construit cadrul conceptual și analitic necesar pentru proiectarea și implementarea portalului InternshipFinder. Parcurgerea sistematică a fundamentelor teoretice relevante - comunicarea web, securitatea stateless, persistența relațională, stocarea în cloud, componentizarea interfeței și internaționalizarea - a demonstrat că fiecare domeniu aduce constrângeri și principii proprii, care trebuie înțelese în profunzime înainte de a fi traduse în decizii de proiectare.

Natura stateless a protocolului de comunicare web impune proiectarea explicită a mecanismelor de autentificare și autorizare. [1] Tokenurile verificabile reprezintă soluția conceptuală potrivită pentru autentificarea stateless, oferind scalabilitate naturală și eliminând nevoia de sesiuni server-side. [2] Modelul relațional cu proprietăți ACID garantează integritatea datelor interdependente, iar stratul de abstractizare a persistenței reduce discrepanța structurală dintre modelul obiectual și cel relațional. [3]

Externalizarea stocării fișierelor către un serviciu cloud specializat separă preocupările corecte și permite scalarea independentă a componentei de stocare. [7] Componentizarea interfeței și mecanismele de gestionare globală a stării sunt esențiale pentru aplicații cu fluxuri complexe și roluri multiple. [5] Internaționalizarea interfeței, prin externalizarea resurselor de traducere, asigură adaptabilitatea platformei la un public divers. [8]

Prin analiza arborelui problemei au fost identificate opt cauze structurale distincte, fiecare cu un mecanism propriu de generare a dificultăților de proiectare. Efectele nerezolvării lor includ vulnerabilități de securitate, inconsistențe de date, costuri crescute de mentenanță și o experiență de utilizare degradată. Obiectivele definite în răspuns direct la aceste cauze vor ghida întregul demers de proiectare și implementare prezentat în capitolele următoare, asigurând că fiecare decizie poate fi conectată la o nevoie identificată și justificată teoretic.

II STUDII DE CAZ ȘI SOLUȚIILE PROPUSE PENTRU INTERNSHIPFINDER

Proiectarea unui sistem software de calitate nu se desfășoară în izolare față de peisajul soluțiilor existente. Un demers de proiectare riguroasă include cunoașterea modului în care problema a fost abordată de alte sisteme, identificarea limitelor acestor abordări și extragerea principiilor și practicilor care au demonstrat valoare în contexte similare. Totodată, alegerea stivei tehnologice nu poate fi redusă la o preferință personală sau la popularitatea momentului - ea trebuie să fie rezultatul unei evaluări comparative bazate pe criterii derivate din cerințele specifice ale sistemului. Capitolul de față urmărește ambele obiective: analiza platformelor reprezentative din piața locală de recrutare și stagii, și evaluarea comparativă a tehnologiilor disponibile pentru fiecare componentă a portalului InternshipFinder.

2.1. Scopul capitolului și metodologia cercetării

Capitolul II are un scop dublu: pe de o parte, să fundamenteze deciziile de proiectare prin analiza comparativă a soluțiilor existente; pe de altă parte, să construiască arborele soluțiilor ca răspuns structurat la cauzele identificate în arborele problemei din capitolul precedent. Metodologia utilizată este exploratorie și comparativă, bazată pe analiza directă a platformelor și pe evaluarea documentației tehnice a tehnologiilor considerate.

Selecția platformelor pentru studiile de caz a urmărit relevanța pentru contextul din Republica Moldova și acoperirea unui spectru de abordări: o platformă matură cu acoperire largă și una mai nouă cu accent pe modernitatea experienței de utilizare. Această diversitate permite identificarea atât a bunelor practici consolidate, cât și a tendințelor emergente în proiectarea portalurilor de recrutare.

Evaluarea stivei tehnologice a urmat un criteriu fundamental: fiecare decizie tehnologică trebuie să fie justificată prin avantajele concrete față de alternativele disponibile, în raport cu cerințele specifice ale sistemului. Această abordare elimină riscul alegerii tehnologiilor pe baza popularității momentului și asigură că ansamblul tehnologic ales este coerent și potrivit problemei abordate.

2.2. Analiza portalurilor de recrutare din Republica Moldova

Piața din Republica Moldova dispune de câteva platforme de recrutare online cu acoperire semnificativă, dar oferta specializată pentru stagii de practică studențești este limitată. Analiza a vizat

două platforme reprezentative, urmărind să identifice atât practicile care merită adoptate, cât și lacunele care justifică construirea unei soluții dedicate.

2.2.1. Rabota.md platformă generalistă de recrutare cu acoperire largă a pieței muncii

Rabota.md este cel mai longeviv portal de recrutare din Republica Moldova, cu o prezență pe piață de peste un deceniu și o bază de date extinsă de CV-uri și oferte de muncă. Platforma acoperă o gamă largă de domenii profesionale și include o secțiune dedicată internship-urilor, accesibilă atât studenților, cât și absolvenților recenti. [14]

Din perspectiva funcționalității, Rabota.md oferă funcții esențiale: căutarea ofertelor după mai multe criterii, posibilitatea de a aplica direct prin platformă, și o zonă de gestionare a candidaturilor pentru recrutori. Interfața bilingvă - română și rusă - reflectă realitatea lingvistică a țării și demonstrează în practică valoarea suportului multi-lingvistic, justificând obiectivul O7 al portalului InternshipFinder. [14]

Cu toate acestea, analiza platformei relevă și limitări semnificative. Portalul este proiectat pentru angajare permanentă, nu pentru internship-uri, și tratează stagiile ca o subcategorie a ofertelor generale, fără un flux dedicat adaptat specificului acestora. Nu există un mecanism integrat de urmărire a statusului candidaturii dincolo de notificarea inițială. Generarea documentelor de candidatură nu este suportată - utilizatorul trebuie să pregătească CV-ul în afara platformei și să îl încarce ca fișier atașat. Iar interfața, deși funcțională, prezintă semne de îmbătrânire și nu oferă experiența fluidă pe care utilizatorii moderni o așteaptă.

Concluzia principală a analizei Rabota.md este că platforma demonstrează viabilitatea unui portal de recrutare online în contextul local, dar nu oferă un model adecvat pentru un portal specializat pe internship-uri studențești. InternshipFinder se poate poziționa ca alternativă specializată, cu un flux integrat specific etapei de debut în carieră, mai adaptat nevoilor și așteptărilor utilizatorilor din segmentul vizat.

2.2.2. Jobber.md platformă modernă orientată spre experiența utilizatorului

Jobber.md reprezintă un val mai recent de portaluri de recrutare locale, cu un accent explicit pe simplitatea vizuală și pe fluiditatea experienței de utilizare. Platforma a câștigat vizibilitate în rândul

tinerilor prin colecțiile tematice de oferte - inclusiv cele destinate candidaților fără experiență anterioară -, și printr-o interfață mai modernă față de platformele mai vechi. [13]

Din perspectiva securității și a verificării identității, Jobber.md a adoptat o practică valoroasă: validarea datelor de contact prin coduri de unică utilizare transmise electronic. Această practică reduce riscul înregistrărilor frauduloase și confirmă că utilizatorul deține controlul datelor de contact declarate. InternshipFinder a integrat o abordare similară în propriul flux de înregistrare, adaptând principiul la contextul specific al platformei.

Sintetizând analiza celor două platforme, se conturează profilul unui portal specializat pe internship-uri studențești care să aducă valoare adăugată față de oferta existentă: flux integrat de la publicarea ofertei la confirmarea participantului, urmărire transparentă a statusului candidaturii, generare de documente direct în platformă, suport multi-lingvistic complet și o interfață modernă, adaptată dispozitivelor mobile. Aceste caracteristici au ghidat direct deciziile de proiectare ale InternshipFinder.

2.3. Stiva tehnologică a InternshipFinder evaluare comparativă și decizii arhitecturale

Selecția stivei tehnologice este una dintre deciziile cu cel mai lung impact asupra unui sistem software. O alegere potrivită reduce fricțiunea de dezvoltare, favorizează calitatea codului și permite extensibilitatea pe termen lung, în timp ce o alegere nepotrivită generează dificultăți care se amplifică odată cu creșterea sistemului. Fiecare componentă a InternshipFinder a fost selectată pe baza unor criterii obiective derivate din cerințele identificate, iar arhitectura de ansamblu reflectă aceste decizii ca un tot coerent.

2.3.1. Arhitectura generală monolit modular cu separare client server

Portalul este structurat ca un monolit modular, cu separarea clară între componenta de server - care gestionează logica domeniului, securitatea și persistența - și componenta de client - care realizează interfața utilizatorului și consumă API-ul serverului. Față de o arhitectură pe microservicii, monolitul modular oferă complexitate operațională semnificativ redusă: un singur proces de server este mai simplu de deploiat, monitorizat și debugat decât zeci de servicii independente. Față de un monolit complet nedivizat, delimitarea clară a modulelor prin interfețe interne bine definite permite refactorizarea și extinderea fiecărui modul fără a afecta celelalte și, pe termen lung, migrarea selectivă a modulelor către servicii independente dacă cerințele de scalare o impun.

2.3.2. Interfața utilizatorului, bibliotecă de randare, stilizare și rutare

React a fost ales ca bibliotecă principală de interfață față de alternative precum Vue sau Angular datorită modelului său declarativ - interfața este descrisă ca o funcție a stării, nu ca o secvență de operații de manipulare -, care produce cod mai ușor de înțeles și de testat. Mecanismul Virtual DOM asigură actualizări selective și eficiente ale interfeței vizibile, menținând fluenta chiar și în scenarii cu modificări frecvente de stare. [5] Arhitectura bazată pe componente funcționale cu stare locală și efecte bine controlate permite construirea aplicațiilor complexe din unități de interfață independente și testabile, reducând complexitatea și facilitând mentenanța pe termen lung. [17]

Interfața este construită ca o aplicație cu o singură pagină (SPA), cu navigare client-side și randare condiționată în funcție de rolul utilizatorului autentificat. Starea de autentificare este gestionată printr-un context global accesibil din orice componentă care trebuie să ia decizii bazate pe identitatea sau rolul utilizatorului curent. React Router gestionează rutarea declarativă a aplicației, inclusiv protecția rutelor pe baza rolului, redirecționând automat utilizatorii neautorizați la fiecare tranziție de rută.

Tailwind CSS, bazat pe clase utilitare, a fost preferat față de abordările tradiționale cu foi de stil personalizate deoarece elimină conflictele CSS frecvente în proiectele de dimensiuni medii și mari și reduce costul cognitiv al înțelegerii aspectului vizual al unei componente - clasele descriu direct proprietățile vizuale, fără nivelul de abstractizare suplimentar al componentelor CSS.

Vite a fost ales ca instrument de construire față de alternativele tradiționale (Webpack, Create React App) datorită utilizării modulelor ES native în faza de dezvoltare, care elimină necesitatea bundling-ului complet la pornire și permite actualizarea modulelor modificate în timp real prin Hot Module Replacement instantaneu, fără a reîncărca întreaga aplicație. [6] Această caracteristică reduce semnificativ ciclul de feedback în dezvoltare. Proxy-ul integrat rezolvă elegant problemele de origine încrucișată în mediul de dezvoltare, redirecționând transparent cererile API către componenta de server - eliminând necesitatea configurării CORS pe server în timpul dezvoltării.

2.3.3. Biblioteci utilitare pentru funcționalități specifice

Gestionarea și formatarea datelor calendaristice este realizată prin date-fns, o bibliotecă cu suport nativ pentru locale, integrată cu mecanismul de internaționalizare al aplicației. Afișarea datelor în formatul corect pentru fiecare limbă suportată - inclusiv ordinea componentelor și denumirile lunilor în română, engleză sau rusă - contribuie la coerența experienței de utilizare.

Generarea documentelor PDF direct în browser, cu ajutorul jsPDF, reprezintă una dintre funcționalitățile distinctive ale platformei față de concurența locală analizată. Componenta CV Builder compune documentul PDF cu datele profilului studentului și îl descarcă automat în browser, fără a necesita instalarea de software suplimentar și fără a trimite datele la server pentru procesare. Această abordare reduce latența percepută, elimină un punct de eșec suplimentar și oferă utilizatorului control direct asupra documentului generat.

react-toastify este utilizat pentru notificările de tip toast, oferind feedback instant utilizatorului în urma acțiunilor efectuate în interfață - aplicare la un stagiu, salvarea profilului, erori de validare.

2.3.4. Componenta de server, cadru de aplicație, securitate și persistență

Java a fost ales pentru performanța JVM-ului modern, pentru stabilitatea suportului pe termen lung și pentru caracteristicile de limbaj care îmbunătățesc lizibilitatea codului - Records pentru DTO-uri concise, Text Blocks pentru interogări sau template-uri HTML.

Spring Boot reduce semnificativ boilerplate-ul de configurare prin auto-configurare și convenție înaintea configurației, accelerând dezvoltarea aplicațiilor enterprise prin eliminarea configurărilor repetitive. [4] Arhitectura pe straturi (controller → service → repository) reduce cuplarea și menține separarea responsabilităților, iar mecanismul de injecție a dependențelor permite decuplarea componentelor și testarea lor izolată, fără dependențe reale față de baza de date sau de serviciile externe.

Spring Security implementează mecanismele de autentificare și autorizare cu suport integrat pentru JWT și politici de acces bazate pe roluri. [4] Autentificarea este realizată prin tokenuri cu durată limitată de viață, transmise clientului prin cookie-uri cu attribute de securitate care previn accesul din codul JavaScript al paginii și limitează transmiterea la conexiuni securizate - alegere preferată față de stocarea tokenurilor în mecanismele de stocare ale browserului, care expun la riscul atacurilor de tip cross-site scripting. [2] Configurarea STATELESS asigură că nicio sesiune server-side nu este creată sau utilizată, în deplină concordanță cu natura stateless a HTTP. [1] Controlul accesului pe roluri este implementat declarativ la nivelul cadrului, politicile de acces fiind centralizate și ușor de auditat, în locul verificărilor disparate în corpul fiecărei metode de control.

Verificarea identității la înregistrare prin cod de unică utilizare - inspirată de practica observată la Jobber.md - adaugă un nivel suplimentar de asigurare că utilizatorul deține controlul adresei electronice declarate, prevenind înregistrările frauduloase. Codul are durată limitată de valabilitate și poate fi utilizat o singură dată, prevenind atacurile prin refolosire. Spring Boot Mail cu JavaMailSender

gestionează trimiterea acestor email-uri tranzacționale, inclusiv pentru recuperarea parolei, prin orice server SMTP configurat.

Spring Data JPA cu Hibernate furnizează stratul de persistență, implementând pattern-ul Repository și abstractizând operațiile de bază de date la nivel de obiecte Java. [3] Datele structurate ale portalului sunt stocate într-o bază de date relațională, aleasă pentru garanțiile sale de integritate și pentru expresivitatea modelului de constrângeri - relațiile dintre entități (utilizatori, profiluri, oferte, candidaturi) sunt exprimate prin chei externe cu constrângeri de integritate referențială, garantând că referințele între entități sunt întotdeauna valide. Repository-urile generează automat implementările pentru operațiile comune - căutare după identificator, paginare, interogări derivate - reducând cantitatea de cod repetitiv. Tranzacțiile sunt gestionate declarativ, prin adnotarea metodelor de serviciu care necesită garanții ACID.

Spring Boot Validation (Jakarta Bean Validation) permite definirea constrângerilor direct pe clasele DTO prin adnotări, prevenind intrarea datelor eronate înainte de a ajunge în stratul de business sau în baza de date. Lombok reduce codul boilerplate: adnotările *@Data*, *@Builder* și variantele lor generează la compilare getteri, setteri, constructori și metode equals/hashCode, sporind lizibilitatea claselor de entitate și DTO. [20]

Migrările versionizate ale schemei bazei de date asigură că modificările structurale sunt aplicate în ordine determinată și pot fi anulate controlat, garantând că schema din baza de date corespunde întotdeauna versiunii codului în execuție - practică esențială în echipe cu mai mulți dezvoltatori sau în sisteme cu cicluri de eliberare frecvente.

2.3.5. Stocarea fișierelor în cloud

Fișierele utilizatorilor - CV-uri și fotografii de profil - sunt stocate exclusiv în Azure Blob Storage, fără a fi persistate în baza de date relațională; aceasta stochează doar referința URL, folosită pentru generarea adreselor de acces la momentul solicitării. [7] Integrarea este realizată prin SDK-ul oficial Java. La upload, backend-ul primește fișierul prin endpoint REST multipart, îl transmite containerului Azure și persistă URL-ul în PostgreSQL; la download, URL-ul este furnizat direct clientului, care accesează fișierul din Azure fără a trece prin backend, eliberând resursele serverului.

Această separare - metadata în baza relațională, conținut binar în cloud - reduce complexitatea schemei relaționale, îmbunătățește performanța interogărilor și permite scalarea independentă a capacității de stocare. [10] Mecanismul de adrese de acces temporare permite ca fișierele private să fie

accesate direct de browser pe durata sesiunii, fără a trece prin serverul aplicației, îmbunătățind performanța percepută și reducând sarcina pe server.

2.3.6. Infrastructura de dezvoltare și producție

Docker Compose standardizează mediul de baze de date, declarând complet serviciile, versiunile, configurațiile și volumele de date într-un fișier de compoziție care poate fi pornit cu o singură comandă pe orice sistem care suportă containerele. [9] Această abordare elimină problema clasică „*funcționează la mine*” și asigură că toți dezvoltatorii lucrează cu aceleași condiții. PostgreSQL este containerizat cu un volum persistent, eliminând necesitatea instalării locale; volumul asigură că datele de test nu se pierd la repornirea containerului.

Configurarea prin variabile de mediu se realizează prin fișierul `.env.properties`, al cărui template este distribuit prin `.env.properties.example`. Credențialele sensibile - parola bazei de date, cheia secretă JWT, connection string Azure, credențialele SMTP - sunt externalizate din codul sursă, urmând principiile de configurare externalizată pentru aplicații cloud-native. [4]

Axios cu `baseURL="/api"` și `withCredentials: true` asigură că toate cererile HTTP includ cookie-urile de autentificare și folosesc prefixul API unificat. Proxy-ul Vite redirecționează în mediul de dezvoltare cererile `/api` către backend-ul Spring Boot, eliminând problemele de CORS fără configurare suplimentară pe server. [6]

2.3.7. Internaționalizarea interfeței

Suportul pentru mai multe limbi este o cerință derivată din realitatea lingvistică a publicului țintă - studenți din Republica Moldova, unde română, rusă și engleză coexistă în mediul academic și profesional. `i18next` și `react-i18next` implementează internaționalizarea cu resurse pentru cele trei limbi și cu selecția limbii salvată în `localStorage`. [8] Hook-ul `useTranslation` setează dinamic titlul paginii și conținutul textelor în funcție de limba curentă, fără a hardcoda niciun text. Organizarea resurselor de traducere în fișiere separate pe limbă, cu o structură ierarhică pe chei, permite actualizarea independentă a fiecărei traduceri și delegarea sarcinii de traducere unor colaboratori fără cunoștințe de programare. Mecanismul de fallback previne afișarea cheilor brute în interfață în cazul traducerilor incomplete.

2.4. Arborele soluțiilor

Arborele soluțiilor este instrumentul prin care deciziile arhitecturale prezentate în secțiunea anterioară sunt conectate explicit la cauzele identificate în arborele problemei. Fiecare soluție este definită în raport cu cauza pe care o adresează, asigurând trasabilitatea completă de la problemă la decizie de proiectare și, ulterior, la implementare.

S0 - Soluția centrală

Soluția centrală constă în proiectarea și implementarea portalului InternshipFinder ca un sistem coerent, securizat și extensibil, în care toate componentele funcționează împreună fără contradicții arhitecturale. Coerența sistemului este asigurată prin trasabilitatea fiecărei decizii de proiectare la o cerință identificată și prin testarea integrată a interacțiunilor dintre componente.

S1 - API orientat pe resurse cu semantică HTTP coerentă

Comunicarea client-server este realizată prin intermediul unui API cu semantică clară: operațiile sunt exprimate prin metodele HTTP corespunzătoare, resursele sunt identificate prin locatori stabili, iar starea operației este comunicată prin coduri de stare cu semnificație prestabilită. Această decizie adresează cauza C1, furnizând mecanismul prin care clientul și serverul pot schimba informații de identitate la fiecare cerere. [1]

S2 - Autentificare stateless și autorizare declarativă pe roluri

Tokenurile verificabile sunt emise la autentificarea cu succes și validate la fiecare cerere protejată, fără consultarea unui registru de sesiuni. Controlul accesului pe roluri este configurat declarativ, centralizat și auditabil. Soluția adresează cauzele C1 și C2, furnizând mecanismul de identificare repetabilă a utilizatorului și restricționarea accesului la resurse pe baza rolului. [2]

S3 și S4 - Persistență relațională cu strat de abstractizare și mediu containerizat

Datele structurate sunt stocate cu garanții ACID, în tabele cu constrângeri de integritate referențială. Stratul de abstractizare a persistenței mapează entitățile obiectuale la tabele și gestionează tranzacțiile declarativ. Mediul de baze de date este containerizat pentru reproductibilitate. Soluțiile adresează cauzele C3, C4 și parțial C7. [3], [9]

S5 - Stocare cloud externalizată pentru fișierele utilizatorilor

Fișierele binare sunt delegate unui serviciu cloud specializat, integrat prin SDK oficial. Baza de date relațională stochează exclusiv referințele. Adresele de acces temporare permit vizualizarea securizată a fișierelor private fără expunere permanentă. Soluția adresează cauza C5. [7]

S6 - Interfață componentizată cu stare globală și navigare protejată

Aplicația de interfață este construită din componente independente, cu starea de autentificare gestionată global și accesibilă în întregul arbore de componente. Navigarea protejată verifică rolul la fiecare tranziție de rută. Reînnoirea automată a tokenului menține sesiunea activă transparent pentru utilizator. Soluția adresează cauza C6. [5]

S7 - Configurarea proxy-ului de dezvoltare și containerizarea serviciilor

Proxy-ul de dezvoltare redirecționează transparent cererile API de la portul clientului la portul serverului, eliminând problemele de origine încrucișată în mediul local. Containerizarea serviciilor auxiliare standardizează mediul de dezvoltare. Soluția adresează cauza C7. [6], [9]

S8 - Internaționalizare cu resurse externalizate și persistarea preferinței

Resursele de traducere sunt organizate în fișiere separate pe limbă, cu o structură ierarhică pe chei. Preferința lingvistică este detectată automat și persistată local. Interpolarea variabilelor în textele traduse asigură corectitudinea gramaticală. Soluția adresează cauza C8. [8]

Pentru mai multă claritate, consultați Tabelul A2.1 din Anexa 2.

2.5. Concluzii la Capitolul II

Capitolul II a fundamentat proiectarea portalului InternshipFinder prin două demersuri complementare și necesare. Analiza comparativă a platformelor Rabota.md și Jobber.md a relevat bunele practici din piața locală - suportul multi-lingvistic, verificarea identității prin coduri de unică utilizare, simplitatea interfeței -, dar și lacunele care justifică construirea unei soluții specializate: absența unui flux integrat de gestionare a internship-urilor, lipsa urmăririi transparente a candidaturilor și absența generării de documente direct în platformă.

Evaluarea comparativă a stivei tehnologice a argumentat că fiecare componentă a sistemului a fost aleasă pe baza unor criterii obiective, derivate direct din cerințele identificate în capitolul anterior. Nicio decizie tehnologică nu a fost luată pe baza preferinței momentului sau a popularității, ci pe baza potrivirii cu cerința specifică pe care trebuie să o satisfacă.

Arborele soluțiilor construiește legătura explicită și trasabilă dintre cauzele identificate în arborele problemei și deciziile arhitecturale concrete care vor ghida implementarea. Această trasabilitate asigură că demersul de proiectare rămâne riguros și justificat pe întreg parcursul lucrării, oferind totodată criteriile față de care soluția implementată poate fi evaluată în capitolul următor.

III PROIECTAREA DETALIATĂ ȘI IMPLEMENTAREA SOLUȚIEI INTERNSHIPFINDER

Capitolele anterioare au stabilit cadrul teoretic, au identificat cauzele problemei, au analizat soluțiile existente și au formulat deciziile arhitecturale care să răspundă obiectivelor cercetării. Capitolul de față transpune toate acestea în cod funcțional, urmărind în detaliu modul în care fiecare componentă a fost concretizată și cum ansamblul lor formează un sistem coerent. Prezentarea urmează logica arhitecturală a soluției: de la configurarea securității și a persistenței, până la interfața utilizatorului, mecanismele de stocare a fișierelor și internaționalizarea interfeței.

3.1. Scopul capitolului și cadrul de implementare

Capitolul III descrie implementarea efectivă a portalului web InternshipFinder, pornind de la fundamentele teoretice și deciziile arhitecturale formulate în Capitolele I-II. Accentul este pus pe modul în care cerințele specifice unui portal multi-rol (student, recruiter, administrator) sunt transpuse în componente software concrete: API REST coerent, autentificare și autorizare stateless, persistență relațională, stocare cloud pentru fișiere, interfață SPA modulară și suport de internaționalizare.

În procesul de implementare a fost urmărită separarea responsabilităților, astfel încât fiecare strat să rămână clar delimitat: controlerele expun contractul HTTP, serviciile implementează regulile de business, repository-urile gestionează persistența, iar componentele de securitate aplică politicile de autentificare și autorizare. Pe partea de client, aplicația React consumă API-ul și implementează rutare protejată, managementul stării de autentificare și selectarea limbii în interfață. În același timp, datele sensibile (chei, parole, connection strings) sunt externalizate în configurare, evitând hardcodarea în codul sursă.

3.2. Structura soluției implementate, client server și servicii auxiliare

Soluția InternshipFinder este realizată în arhitectură client-server, în care backend-ul este construit cu Spring Boot 3 pe Java 21, iar frontend-ul este o aplicație SPA React construită cu Vite. Comunicarea dintre client și server se face prin endpoint-uri REST sub prefixul */api*, conform principiilor HTTP și contractelor stabile de tip cerere/răspuns [1]. Persistența este asigurată de o bază de date relațională PostgreSQL, accesată prin Spring Data JPA (Hibernate) [3], iar fișierele (CV-uri și fotografii

de profil) sunt stocate în Azure Blob Storage, utilizând SDK-ul Java [7]. În Figura 3.1 este prezentată reprezentarea grafică a modelului client-server, evidențiind componentele principale ale sistemului.

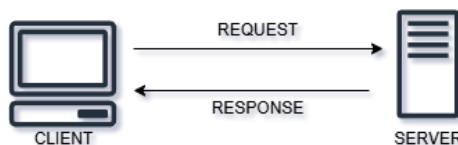


Figura 3.1. Reprezentarea grafică a modelului client-server

În timpul dezvoltării, mediul de baze de date este standardizat prin Docker Compose, reducând variațiile între medii și asigurând reproductibilitatea. În plus, integrarea frontend-backend este simplificată prin proxy-ul Vite care redirecționează cererile */api* către server, eliminând fricțiunea tipică legată de CORS în timpul dezvoltării [6].

3.3. Configurarea aplicației și dependențele principale

Backend-ul utilizează dependențe specifice aplicațiilor enterprise: web (Spring MVC), securitate (Spring Security), validare (Bean Validation), persistență (JPA + PostgreSQL), email (Spring Mail), integrare cloud (Azure Blob Storage) și bibliotecă JWT (jjwt). Alegerea acestor module reflectă direct obiectivele și soluțiile propuse în Capitolul II: securitate stateless, persistența ACID, stocare externalizată a fișierelor și interfață modernă. În Figura 3.2 este prezentată structura fișierului *pom.xml* și modul de interacțiune al acestuia cu ecosistemul Maven, evidențiind gestionarea dependențelor, configurarea proiectului și procesul de automatizare a compilării și distribuirii aplicației. Pentru codul complet al fișierului *pom.xml*, consultați Anexa 3. Listarea A3.1.

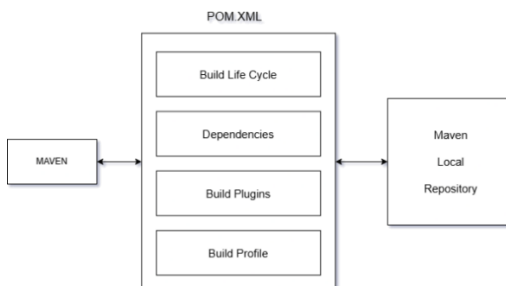


Figura 3.2. Structura fișierului *pom.xml* și interacțiunea cu ecosistemul Maven

Frontend-ul utilizează dependențe specifice aplicațiilor SPA moderne: framework-ul React pentru interfață componentială, Vite pentru build rapid și development server, React Router pentru navigare și rutare protejată, Axios pentru comunicare HTTP cu backend-ul, react-hook-form și

yup pentru gestionarea și validarea formulelor, i18next și react-i18next pentru internaționalizare, jsPDF pentru generarea documentelor PDF în browser, TailwindCSS pentru stilizare utilitară, precum și biblioteci auxiliare pentru iconuri (Lucide React, Heroicons), notificări (react-toastify), selectare date (react-datepicker, date-fns) și securitate DOM (DOMPurify). Alegerea acestor module reflectă direct obiectivele și soluțiile propuse în Capitolul II: interfață reactivă și modernă, validare client-side, suport multilingv și generare locală de documente. Pentru codul complet al fișierului *package.json*, consultați Anexa 3. Listarea A3.2.

Configurația aplicației este externalizată prin *application.properties* și un fișier *.env.properties* (importat opțional), astfel încât setările pentru DB, JWT, SMTP și Azure să nu fie incluse în cod. Această practică este conformă cu principiile recomandate pentru aplicații Spring Boot și reduce riscul expunerii credențialelor [4]. Figura 3.3 ilustrează mecanismul de externalizare a configurațiilor sensibile prin utilizarea variabilelor de mediu, metodă ce contribuie la creșterea securității aplicației prin separarea datelor confidențiale de codul sursă și facilitarea administrării configurațiilor în diferite medii de execuție. Pentru codul complet al fișierului *application.properties*, consultați Anexa 4, Listarea A4.1.

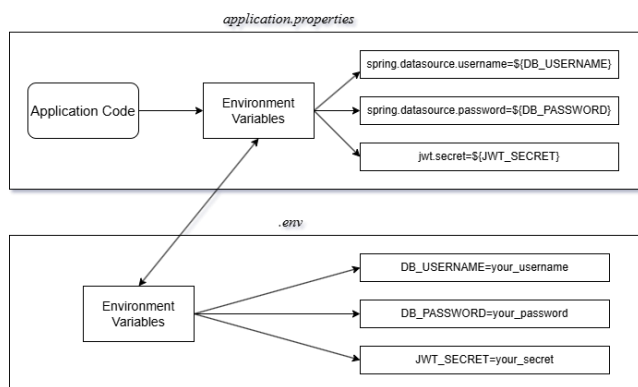


Figura 3.3. Mecanismul de externalizare a configurațiilor sensibile utilizând variabile de mediu

3.4. Implementarea securității stateless, JWT și Spring Security

Conform constrângerilor impuse de HTTP (stateless) [1], autentificarea trebuie să permită serverului să recunoască identitatea utilizatorului la fiecare cerere, fără sesiuni server-side. În InternshipFinder, această cerință este satisfăcută prin utilizarea token-urilor JWT standardizate [2], validate la fiecare request. Controlul accesului este aplicat la nivel de endpoint prin Spring Security, iar aplicația este configurată explicit în mod STATELESS, astfel încât serverul să nu creeze sesiuni. În

Figura 3.4 este prezentată arhitectura sistemului de filtrare și delegare a securității în cadrul Spring Security, evidențiind fluxul de procesare al cererilor, lanțul de filtre utilizat pentru autentificare și autorizare, precum și mecanismul de delegare către componentele responsabile de gestionarea securității aplicației. Pentru configurația *SecurityConfig* (politici de acces și stateless), consultați Anexa 5, Listarea A5.1.

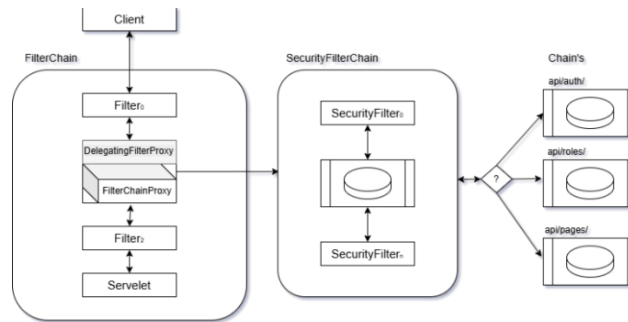


Figura 3.4. Arhitectura sistemului de filtrare și delegare a securității în Spring Security

Generarea token-urilor access este realizată în *JwtTokenProvider*. Token-ul include sub (email) și un claim typ utilizat pentru a distinge tipul token-ului în validare. Semnarea se realizează cu algoritmul HS512, iar cheia secretă este citită din configurare și decodificată Base64. Pentru implementarea *JwtTokenProvider* consultați Anexa 6, Listarea A6.1.

Aplicarea autentificării la runtime se face prin filtrul *JwtAuthenticationFilter*. Filtrul extrage token-ul din cookie-ul *accessToken*, validează token-ul și, dacă acesta este valid, încarcă *UserDetails* și setează contextul de securitate. Această abordare păstrează backend-ul stateless și compatibil cu principiul că fiecare cerere trebuie să conțină informații suficiente pentru autentificare [1], [2]. Figura 3.5 prezintă diagrama de secvență a procesului de reînnoire a token-ului de acces, evidențiind etapele de validare a token-ului de reîmprospătare, generarea unui nou token de acces și comunicarea dintre client și componentele responsabile de autentificare și autorizare. Pentru implementarea *JwtAuthenticationFilter* consultați Listarea A6.2, din Anexa 6.

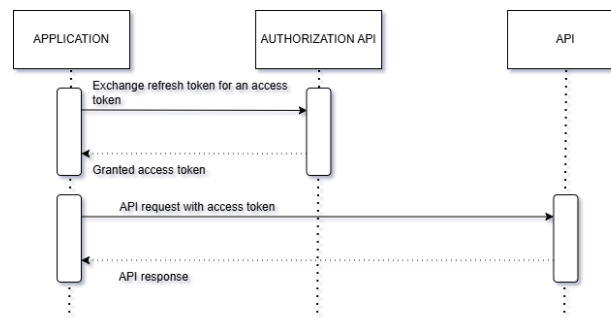


Figura 3.5. Diagrama de secvență a procesului de reînnoire a token-ului de acces

3.5. Implementarea fluxurilor de autentificare, reîmprospătare și parolă unică

API-ul de autentificare este concentrat în `AuthController` sub ruta `/api/auth`. Implementarea acoperă înregistrarea, autentificarea, logout-ul și refresh-ul token-ului de acces. În plus, este implementat un mecanism OTP (One-Time Password) pentru operații sensibile, precum resetarea parolei și verificarea adresei de email, aspect inspirat din practicile observate în platformele analizate în Capitolul II. Figura 3.6 ilustrează mecanismul de autentificare al utilizatorului și gestionarea sesiunilor prin Access Token și Refresh Token.



Figura 3.6. Gestiunea securizată a sesiunilor prin utilizarea Access Token și Refresh Token

Sistemul de autentificare este stateless, neconservând sesiuni în baza de date, ci doar refresh token-uri pentru revocare centralizată. Access token-ul JWT, cu durată scurtă, este validat la fiecare cerere prin semnătură HS512 și claim-ul type: "access". La expirare, clientul trimite refresh token-ul din cookie către `/auth/refresh`, iar serverul verifică validitatea și generează un nou access token. Această separație reduce riscul de compromitere, access token-ul fiind de scurtă durată, iar refresh token-ul putând fi invalidat centralizat. Figura 3.7 prezintă fluxul de autorizare a cererilor utilizatorului prin intermediul API-ului de securitate, evidențiind verificarea Access Token-ului, validarea permisiunilor și controlul accesului la resursele protejate înainte de procesarea cererii. Pentru fragmentul de login din `AuthController`, consultați Anexa 7, Listarea A7.1 și Anexa 8, Listarea A8.2 pentru `AuthContext.jsx`.

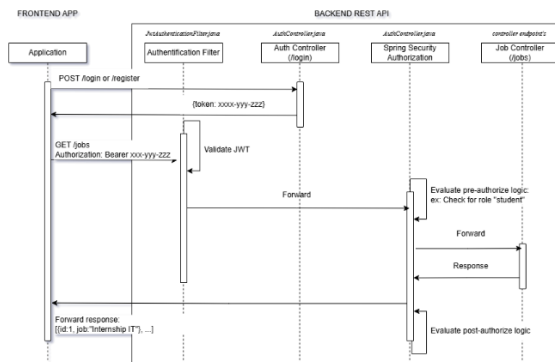


Figura 3.7. Fluxul de autorizare a cererilor utilizatorului prin intermediul API-ului de securitate

OTP-urile sunt persistate într-o entitate JPA dedicată (OtpToken), ceea ce permite audit, expirare și prevenirea reutilizării. Modelul include purpose pentru a separa cazurile de utilizare (resetare parolă, verificare email student/recruiter, schimbare email etc.), plus timestamp-uri pentru expirare și consum. Figura 3.8 prezintă diagrama de secvență a procesului de verificare prin OTP, evidențiind generarea codului, transmiterea acestuia către utilizator și validarea sa de către sistem pentru confirmarea autentificării. Pentru codul complet al fișierului *OtpToken.java*, consultați Anexa 9, Listarea A9.1.

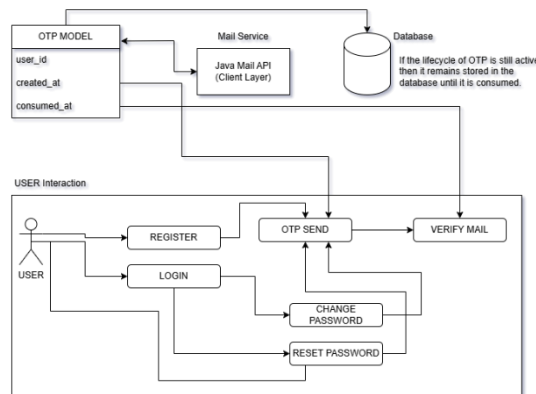


Figura 3.8. Diagrama de secvență a procesului de verificare prin OTP

Trimiterea efectivă a OTP către utilizator este realizată prin Spring Mail. Implementarea verifică existența credențialelor SMTP și generează email-uri HTML pentru o experiență mai prietenoasă. Acest mecanism completează partea de securitate, deoarece adaugă o verificare suplimentară pentru acțiuni sensibile și înregistrări. Figura 3.9 ilustrează fluxul de comunicare dintre clientul Java și serverul SMTP, evidențiind procesul de trimitere a mesajelor de e-mail, autentificarea pe server și livrarea conținutului către destinatar. Pentru fragmentul de trimitere a OTP din *EmailServiceImpl*, consultați Anexa 9, Listarea A9.2.

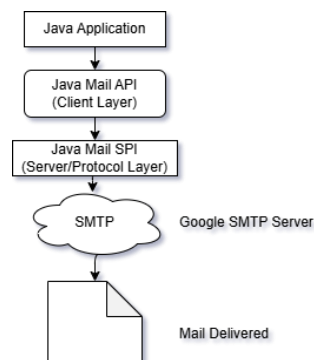


Figura 3.9. Fluxul de comunicare între clientul Java și serverul SMTP

3.6. Persistența PostgreSQL și standardizarea mediului de dezvoltare

Persistența relațională este realizată cu PostgreSQL, aleasă pentru suportul robust al tranzacțiilor ACID și al integrității referențiale [3]. Pentru a asigura un mediu reproductibil, baza de date rulează prin Docker Compose. Această decizie reduce dependențele locale și permite pornirea rapidă a proiectului pe un sistem nou, menținând același comportament în dezvoltare. Figura 3.10 ilustrează configurarea serviciului PostgreSQL în Docker Compose, evidențiind parametrii de rulare ai containerului, variabilele de mediu și maparea porturilor utilizate pentru conectarea la baza de date.

```
version: "3.9"

services:
  postgres:
    image: postgres:16
    container_name: internship-finder-postgres
    environment:
      POSTGRES_DB: internship_finder
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Figura 3.10. Definirea containerului PostgreSQL în fișierul Docker Compose

3.7. Implementarea stocării fișierelor în Azure Blob Storage

O cerință practică importantă a portalului este gestionarea fișierelor încărcate de utilizatori, în special CV-uri și fotografii de profil. Stocarea acestor fișiere direct în baza de date ar crea costuri și probleme de performanță, de aceea a fost aleasă externalizarea către Azure Blob Storage, conform principiilor descrise în Capitolul I [7]. Integrarea este realizată printr-un bean Spring care construiește BlobContainerClient și asigură existența containerului. În Figura 3.11 este prezentată configurarea clientului BlobContainerClient, utilizat pentru inițializarea conexiunii cu Azure Blob Storage și accesarea containerelor de stocare.

```
@Bean
public BlobContainerClient blobContainerClient() {
    String connectionString = getRequiredSetting("azure.storage.connection-string", "AZURE_STORAGE_CONNECTION_STRING");
    String containerName = getRequiredSetting("azure.storage.container-name", "AZURE_STORAGE_CONTAINER_NAME");
    BlobServiceClient blobServiceClient = new BlobServiceClientBuilder()
        .connectionString(connectionString)
        .buildClient();

    BlobContainerClient containerClient = blobServiceClient.getBlobContainerClient(containerName);
    if (!containerClient.exists()) {
        containerClient.create();
    }
    return containerClient;
}
```

Figura 3.11. Implementarea Bean-ului pentru accesul la Azure BlobContainerClient

În Figura 3.12 este prezentat fluxul de date pentru procesarea documentelor în cadrul aplicației. Operația de upload este implementată în clasa FileUploadServiceImpl, care generează un nume unic pentru fiecare fișier prin combinarea folderului logic, a unui timestamp și a unui UUID. După încărcare, se configurează tipul de conținut (*Content-Type*), iar URL-ul rezultat este stocat ca referință. Pentru acces controlat poate fi generat un URL SAS temporar cu perioadă de valabilitate configurabilă. Implementarea completă a clasei este prezentată în Anexa 7, Listarea A7.2.

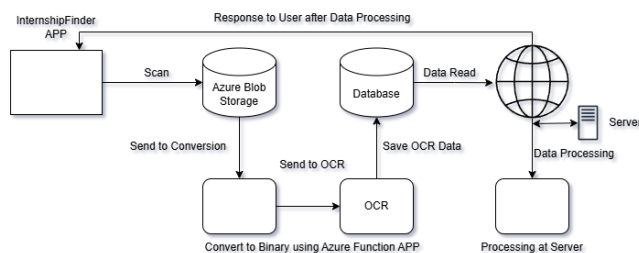


Figura 3.12. Fluxul de date pentru procesarea documentelor în cadrul aplicației

3.8. Implementarea interfeței de programare a aplicației pe roluri

Portalul este proiectat ca aplicație multi-rol, iar acest lucru se reflectă direct în structura endpoint-urilor. Rutele sunt segmentate sub */api/student*, */api/recruiter* și */api/admin*, iar Spring Security aplică restricțiile. Astfel, chiar dacă UI ascunde anumite funcționalități, securitatea rămâne impusă autoritativ în backend, conform bunelor practici discutate în fundamentarea teoretică [4].

În zona studentului, sunt expuse endpoint-uri pentru obținerea și actualizarea profilului, plus upload de CV și fotografie de profil. Implementarea utilizează *MultipartFile* și delegă către serviciu logic de stocare în Azure, păstrând controller-ul subțire. A se vedea Anexa 10, Listarea A10.1 pentru un exemplu complet.

Recruiter-ului, API-ul permite postarea și administrarea joburilor în contextul utilizatorului autentificat. Identitatea recruiter-ului este obținută prin *Principal*, rezultat din autentificarea aplicată de filtrul JWT. A se vedea Anexa 10, Listarea A10.2 pentru un exemplu complet.

Zona administratorului consolidează funcții de management și control: listarea utilizatorilor, actualizarea profilurilor, blocarea/activarea conturilor, ștergerea utilizatorilor, gestionarea joburilor și colectarea logurilor. O particularitate importantă este integrarea logării acțiunilor administrative, ceea ce contribuie la trasabilitate și audit intern. Figura 3.13 ilustrează implementarea controlului accesului bazat pe roluri (RBAC), evidențiind modul în care drepturile utilizatorilor sunt asociate rolurilor și utilizate

pentru restricționarea accesului la resursele aplicației. A se vedea Anexa 10, Listarea A10.3 pentru un exemplu complet.

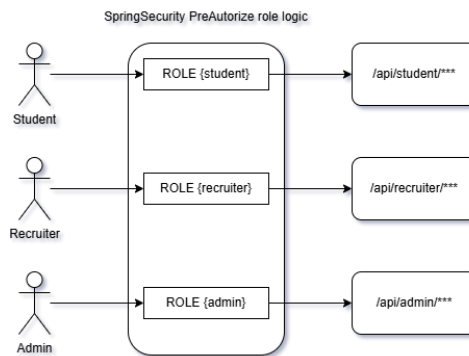


Figura 3.13. Implementarea controlului accesului bazat pe roluri (RBAC)

3.9. Implementarea interfeței utilizatorului, integrare, sesiune și rutare protejată

Frontend-ul este realizat ca aplicație SPA în React, ceea ce permite navigare rapidă fără reîncărcarea paginii și implementarea unor fluxuri specifice portalurilor (dashboard-uri pe roluri, pagini protejate, form-uri dinamice). Pentru development, Vite asigură pornire rapidă și integrare simplă a proxy-ului către backend [6]. Astfel, cererile către `/api` sunt direcționate către serverul Spring Boot, fără a necesita configurări suplimentare de CORS la fiecare schimbare. A se consulta Anexa 11, Listarea A11.1.

Consumarea API-ului se face printr-o instanță Axios configurată cu `baseURL="/api"` și `withCredentials=true`, pentru a permite trimiterea cookie-urilor de autentificare la fiecare request. A se consulta Anexa 11, Listarea A11.2.

Accesul la pagini este protejat prin componenta `ProtectedRoute`, care verifică existența utilizatorului autentificat și rolul acestuia. A se consulta Anexa 8, Listarea A8.1 pentru fișierul `ProtectedRoute.jsx`. Deși această protecție este la nivel de UI, ea este menită să ofere o experiență corectă utilizatorului; securitatea reală rămâne impusă pe backend prin Spring Security [4]. A se consulta Anexa 11, Listarea A11.3.

Managementul sesiunii este implementat în `AuthContext`. La încărcarea aplicației, clientul încearcă să obțină sesiunea curentă prin GET `/auth/me`. Dacă serverul răspunde cu 401/403 (token expirat sau absent), un interceptor Axios încearcă să execute POST `/auth/refresh`. Dacă refresh-ul reușește, request-ul inițial este reluat; în caz contrar, utilizatorul este delogat. Acest model reduce frecvența re-autentificării și susține o experiență consistentă într-un sistem stateless. Figura 3.14 ilustrează diagrama

fluxului de date între client și server prin intermediul REST API, evidențiind modul de transmitere și procesare a cererilor și răspunsurilor în cadrul comunicației dintre componentele sistemului. A se consulta Anexa 11, Listarea A11.4.



Figura 3.14. Diagrama fluxului de date între client și server via REST API

3.10. Implementarea funcționalității CV Builder

Funcționalitatea CV Builder a fost implementată pentru a permite studentului să își creeze un CV direct în aplicație și să îl exporte în format PDF, fără a depinde de un serviciu server-side de generare a documentelor. Această abordare reduce încărcarea backend-ului și oferă feedback imediat utilizatorului, deoarece documentul este generat local în browser prin biblioteca jsPDF. Modulul este integrat în aplicația React ca pagină separată și este accesibil doar utilizatorilor cu rolul STUDENT, prin mecanismul de rutare protejată.

În App.jsx, ruta către CV Builder este declarată protejată, astfel încât pagina să fie accesibilă doar studentului autentificat.

Implementarea principală se află în CvBuilder.jsx, unde datele sunt colectate prin react-hook-form și validate prin yup, schema fiind generată în cvBuilderSchema.js. Validarea este adaptată secțiunilor opționale (de exemplu experiență, proiecte, certificări), astfel încât utilizatorul să poată dezactiva o secțiune fără a fi blocat de erori. De exemplu, secțiunea „*experience*” devine obligatorie doar dacă este activată. A se vedea Anexa 12, Listarea A12.1 și Listarea A12.5.

Generarea PDF-ului este realizată în CvBuilder.jsx prin construirea unui obiect jsPDF, compunerea conținutului pe secțiuni și exportul local al fișierului prin pdf.save(...). A se vedea Anexa 12, Listarea A12.3.

Un aspect important al implementării este suportul pentru caractere Unicode (diacritice și alfabet chirilic), necesar datorită interfeței multilingve. Pentru aceasta, în CvBuilder.jsx sunt încărcate fonturi *TTF (Noto Sans)* și încorporate în document, cu fallback la fonturile implicite ale jsPDF dacă încărcarea nu reușește. A se vedea Anexa 12, Listarea A12.2.

După generare, aplicația oferă și salvarea CV-ului ca document oficial al studentului, încărcând PDF-ul rezultat către backend pe endpoint-ul de upload al CV-ului. În CvBuilder.jsx, PDF-ul este

convertit în Blob, ambalat într-un File și transmis ca *multipart/form-data*, ceea ce unifică fluxul cu mecanismul standard de gestionare a fișierelor. A se vedea Anexa 12, Listarea A12.4.

Figura 3.15 ilustrează arhitectura funcțională a sistemului de generare a CV-ului (CV Builder), evidențiind principalele componente ale aplicației și modul în care acestea interacționează pentru a permite crearea, procesarea și exportul CV-urilor.

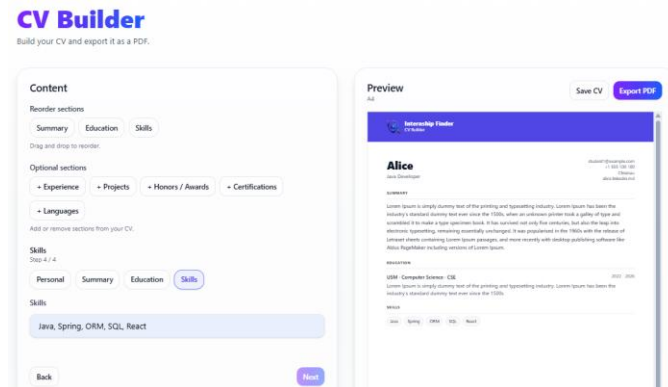


Figura 3.15. Arhitectura funcțională a sistemului de generare a CV-ului (CV Builder)

3.11. Internaționalizarea și persistarea preferinței de limbă

Conform obiectivului de internaționalizare, aplicația suportă trei limbi (română, engleză, rusă) prin *i18next* și integrarea *react-i18next* [8]. Resursele sunt încărcate local, iar limba selectată este păstrată în *localStorage*, astfel încât preferința să fie menținută la reîncărcarea aplicației. Această abordare elimină hardcodarea textelor și permite extinderea ulterioară prin adăugarea de noi fișiere de traducere. Figura 3.16 ilustrează diagrama fluxului de interacțiune a utilizatorului cu modulul de internaționalizare (*i18n*), evidențiind modul în care aplicația gestionează schimbarea limbii și încărcarea dinamică a resurselor lingvistice în funcție de preferințele utilizatorului.

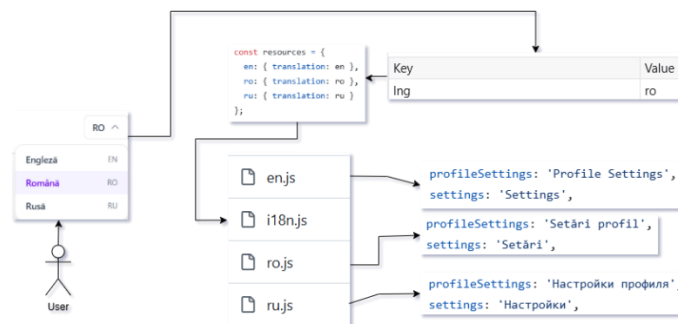


Figura 3.16. Diagrama fluxului de interacțiune a utilizatorului cu modulul de internaționalizare (i18n)

3.12. Concluzii la Capitolul III

Implementarea InternshipFinder confirmă practic ipotezele și deciziile formulate în Capitolele I-II. Securitatea stateless a fost realizată prin JWT și Spring Security, aplicând politici de acces pe roluri și validând identitatea utilizatorului la fiecare cerere fără sesiuni server-side [1], [2], [4]. Persistența în PostgreSQL este integrată prin Spring Data JPA, iar mediul este standardizat prin Docker Compose, ceea ce permite reproductibilitate și stabilitate în dezvoltare [3], [9]. Gestionarea fișierelor a fost separată de baza de date prin integrarea Azure Blob Storage, crescând scalabilitatea și reducând încărcarea componentelor de persistență [7]. Funcționalitatea CV Builder, implementată integral în client prin biblioteca jsPDF, demonstrează capacitatea aplicației de a oferi instrumente utile direct în browser, fără a depinde de servicii backend suplimentare, facilitând generarea documentelor standardizate pentru studenți. La nivel de interfață, SPA-ul React, rutarea protejată și mecanismul de refresh al sesiunii oferă o experiență coerentă utilizatorilor, iar internaționalizarea prin i18next asigură adaptarea la utilizatori cu preferințe lingvistice diferite [5], [6], [8].

Prin urmare, capitolul demonstrează că portalul a fost implementat ca un sistem coerent și extensibil, în care fiecare subcomponentă răspunde direct unei cauze identificate în arborele problemei și unei soluții propuse în arborele soluțiilor.

CONCLUZII

Lucrarea de față a urmărit, de la prima până la ultima pagină, o logică progresivă și coerentă: înțelegerea profundă a problemei înainte de propunerea oricărei soluții, justificarea riguroasă a fiecărei decizii de proiectare și demonstrarea practică că deciziile luate produc un sistem funcțional. Această abordare nu este o simplă convenție academică, ci reflectă o convingere fundamentală despre calitatea în ingineria software: un sistem bun pornește de la o problemă bine înțeleasă, nu de la o tehnologie pre ferată.

Analiza fundamentelor teoretice a revelat că provocările unui portal web multi-rol modern sunt profunde și interconectate. Natura stateless a protocolului de comunicare web nu este o limitare accidentală, ci o proprietate de design care conferă scalabilitate și care impune proiectarea explicită a mecanismelor de autentificare. Modelul relațional de stocare a datelor nu este o alegere arbitrară, ci expresia unui sistem formal cu garanții matematice de integritate, esențiale în aplicații în care datele sunt interdependente și accesate concurrent. Componentizarea interfeței nu este o modă pasageră, ci un principiu de inginerie software cu decenii de validare practică, aplicat la contextul specific al aplicațiilor web.

Analiza platformelor existente din piața locală a confirmat că există un spațiu neacoperit pentru un portal specializat pe internship-uri studențești. Platformele generalist existente acoperă parțial nevoia, dar nu oferă un flux integrat adaptat specificului etapei de debut în carieră. InternshipFinder răspunde acestui spațiu cu un set de funcționalități gândit explicit pentru cele trei categorii de utilizatori - studenți, recrutori și administratori -, fiecare cu propriul context funcțional și propriile cerințe de securitate și experiență de utilizare.

Implementarea a demonstrat că arhitectura propusă este funcțională și că deciziile de proiectare luate în faza de analiză sunt validate în practică. Sistemul de securitate stateless funcționează corect în scenarii de expirare și reînnoire a tokenurilor. Persistența relațională menține integritatea datelor în condiții de acces concurrent. Stocarea externalizată a fișierelor reduce corect sarcina pe baza de date și permite scalarea independentă. Interfața componentizată cu navigare protejată oferă o experiență coerentă și adaptată fiecărui rol de utilizator.

Direcțiile de extindere ale portalului sunt numeroase și bine definite de arhitectura extensibilă aleasă. Adăugarea unui modul de recomandare a ofertelor, bazat pe analiza profilului studentului, ar crește relevanța rezultatelor de căutare. Extinderea suportului lingvistic și integrarea cu sisteme externe de calendar sau de management al talentelor ar crește valoarea platformei pentru organizații mai mari.

În concluzie, lucrarea demonstrează că construirea unui sistem informatic modern, securizat și extensibil este, înainte de toate, un exercițiu de gândire sistematică. Identificarea corectă a problemei, analiza riguroasă a cauzelor sale, formularea precisă a obiectivelor, evaluarea comparativă a soluțiilor și implementarea coerentă a deciziilor luate - acestea sunt elementele care transformă o colecție de tehnologii într-un sistem cu valoare reală pentru utilizatorii săi.

BIBLIOGRAFIE

- [1] FIELDING, R. et al. RFC 9110: HTTP Semantics. RFC Editor, 2022. Disponibil la: <https://www.rfc-editor.org/rfc/rfc9110.html> [Accesat la 10.02.2025]
- [2] JONES, M.; BRADLEY, J.; SAKIMURA, N. RFC 7519: JSON Web Token (JWT). RFC Editor, 2015. Disponibil la: <https://www.rfc-editor.org/rfc/rfc7519> [Accesat la 10.02.2025]
- [3] PostgreSQL Global Development Group. PostgreSQL Documentation - Concepts. Disponibil la: <https://www.postgresql.org/docs/current/tutorial-concepts.html> [Accesat la 10.02.2025]
- [4] VMware. Spring Boot Reference Documentation. Disponibil la: <https://docs.spring.io/spring-boot/reference/> [Accesat la 10.02.2025]
- [5] Meta. React Documentation - Describing the UI. Disponibil la: <https://react.dev/learn/describing-the-ui> [Accesat la 10.02.2025]
- [6] Vite Team. Vite Documentation - Server Options. Disponibil la: <https://vite.dev/config/server-options> [Accesat la 10.02.2025]
- [7] Microsoft Corporation. Azure Blob Storage Documentation - Introduction to Azure Blob Storage. Disponibil la: <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction> [Accesat la 10.02.2025]
- [8] i18next Contributors. i18next Documentation. Disponibil la: <https://www.i18next.com/> [Accesat la 14.02.2025]
- [9] Docker. Docker Compose Documentation. Disponibil la: <https://docs.docker.com/compose/> [Accesat la 10.02.2025]
- [10] NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2nd ed. Sebastopol: O'Reilly Media, 2021. 612 p. ISBN 978-1-492-03402-5
- [11] ELMASRI, Ramez; NAVATHE, Shamkant B. Fundamentals of Database Systems. 7th ed. Hoboken: Pearson, 2017. 1272 p. ISBN 978-0-133-46518-0
- [12] RICHARDSON, Chris. Microservices Patterns: With Examples in Java. Shelter Island: Manning Publications, 2018. 520 p. ISBN 978-1-617-29454-1
- [13] Jobber.md. Portalul de joburi Jobber.md. Disponibil la: <https://jobber.md/> [Accesat la 10.02.2025]

[14] Rabota.md. Portal de Recrutare Moldova. Disponibil la: <https://www.rabota.md/> [Accesat la 10.02.2025]

[15] Rabota.md. Internship-uri și joburi pentru studenți. Disponibil la: <https://www.rabota.md/ro/vacancies/category/internship> [Accesat la 10.02.2025]

[16] BANKS, Alex; PORCELLO, Eve. Learning React: Modern Patterns for Developing React Apps. 2nd ed. Sebastopol: O'Reilly Media, 2020. 310 p. ISBN 978-1-492-05172-5

[17] WALLS, Craig. Spring in Action. 6th ed. Shelter Island: Manning Publications, 2022. 520 p. ISBN 978-1-617-29757-3

[18] BLOCH, Joshua. Effective Java. 3rd ed. Boston: Addison-Wesley Professional, 2018. 412 p. ISBN 978-0-134-68599-1

[19] BRAGARU, Tudor; UNGUREANU, Valeriu. Ghid metodic pentru elaborarea tezelor de an, de licență și de master. Chișinău: Editura USM, 2024. 53 p. ISBN 978-9975-62-686-6

ANEXE

Anexa 1. Tabelele problemei de cercetare

Tabel A1.1. Cauzele determinante ale problemei analizate și originea lor structural

<i>Cauza</i>	<i>Probleme identificate</i>
C1	Constrângerile impuse de caracterul stateless al comunicării prin HTTP: serverul nu reține contextul utilizatorului între cereri consecutive, necesitând proiectarea explicită a unui mecanism de identificare repetabilă. [1]
C2	Necesitatea securizării diferențiate a accesului într-o aplicație cu roluri multiple: în absența unui mecanism de autorizare granulară, resursele pot fi accesate de utilizatori fără drepturile corespunzătoare.
C3	Complexitatea persistenței datelor interdependente în modelul relațional: relațiile dintre entități și constrângerile de integritate impun o proiectare riguroasă a schemei bazei de date. [3]
C4	Discrepanța structurală dintre modelul orientat pe obiecte al aplicației și modelul relațional al bazei de date: maparea manuală a acestei discrepanțe generează cod fragil și dificil de menținut.
C5	Gestionarea ineficientă a fișierelor binare în baza de date relațională: volumele mari de date binare degradează performanța operațiilor de interogare și complică scalarea sistemului. [7]
C6	Complexitatea interfeței unui portal cu multiple roluri și fluxuri de navigare: starea aplicației trebuie să fie gestionată consistent și accesibilă în locurile potrivite ale ierarhiei de componente. [5]
C7	Probleme de integrare între componenta de client și componenta de server în mediul de dezvoltare: politicile de origine încrucișată ale browserelor blochează cererile dacă nu sunt configurate corect.
C8	Necesitatea adaptării interfeței la preferințele lingvistice ale utilizatorilor: un portal destinat unui public divers trebuie să suporte multiple limbi fără a impune rescrierea codului de interfață. [8]

Tabel A1.2. Obiectivele cercetării și corespondența lor cu cauzele identificate

<i>Obiectiv</i>	<i>Descriere și legătura cu cauzele identificate</i>
O1	Realizarea unei arhitecturi client-server cu separarea clară a responsabilităților și comunicare prin API orientat pe resurse cu semantică HTTP coerentă. Adresează C1 prin stabilirea unui contract clar de comunicare. [1]
O2	Implementarea autentificării și autorizării stateless prin tokenuri verificabile cu durată limitată de viață și control al accesului declarativ pe roluri. Adresează C1 și C2. [2]
O3	Stocarea datelor structurate într-un model relațional cu constrângeri de integritate și utilizarea unui strat de abstractizare a persistenței pentru accesul la date. Adresează C3 și C4. [3]
O4	Integrarea unui serviciu cloud dedicat pentru stocarea externalizată a fișierelor utilizatorilor. [7]
O5	Dezvoltarea unei interfețe bazate pe componente cu navigare protejată pe roluri și gestionare globală a stării de autentificare. [5]
O6	Asigurarea comunicării funcționale client-server în mediul de dezvoltare prin configurarea corectă a proxy-ului și a politicilor de origine. [6]
O7	Implementarea internaționalizării interfeței cu resurse externalizate și persistarea preferinței lingvistice a utilizatorului. [8]

Anexa 2. Tabelele soluțiilor propuse

Tabelul A2.1. Corespondența dintre cauzele problemei, soluțiile propuse și tehnologiile utilizate

<i>Cauză (C)</i>	<i>Soluție (S)</i>	<i>Tehnologie concretă</i>
<i>[C1] Constrângerile impuse de comunicarea stateless prin HTTP</i>	<i>[S1] API REST coerent</i>	Spring MVC, Axios, proxy Vite [1], [6]
<i>[C2] Necesitatea securizării accesului într-o aplicație multi-rol</i>	<i>[S2] JWT, Spring Security și OTP</i>	jjwt, SecurityConfig, Spring Mail [2], [4]
<i>[C3] Complexitatea persistenței în modelul relațional</i>	<i>[S3] PostgreSQL cu constrângeri</i>	PostgreSQL 16, Docker Compose [3], [9]
<i>[C4] Diferența structurală dintre modelul orientat pe obiecte și cel relațional</i>	<i>[S4] ORM prin JPA/Hibernate</i>	pring Data JPA, Hibernate, Lombok [3]
<i>[C5] Gestionarea ineficientă a fișierelor binare în baza de date</i>	<i>[S5] Azure Blob Storage</i>	azure-storage-blob SDK, BlobContainerClient [7]
<i>[C6] Complexitatea interfeței și a stării aplicației în client</i>	<i>[S6] SPA React modular</i>	React 19, React Router 7, Tailwind, Toastify [5]
<i>[C7] Probleme de integrare în mediul de dezvoltare (CORS, proxy)</i>	<i>[S7] Docker, Vite proxy și .env</i>	Docker Compose, vite.config.js, env files [9]
<i>[C8] Necesitatea adaptării interfeței la utilizatori cu limbi diferite</i>	<i>[S8] i18next, jsPDF și date-fns</i>	i18next 25, react-i18next, jsPDF, date-fns [8]

Anexa 3. Configurația Maven și dependențele proiectului

```
<properties>
  <java.version>21</java.version>
</properties>

<dependencies>
  <!-- Core & Security -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- Database & Utilities -->
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

Listarea A3.1. Fragment din fișierul pom.xml

```
{
  "name": "client",
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "axios": "^1.13.2",
    "react": "^19.2.3",
    "react-dom": "^19.2.3",
    "react-router-dom": "^7.12.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^5.1.2",
    "vite": "^7.3.1"
  }
}
```

Listarea A3.2. Fragment din fișierul package.json

Anexa 4. Configurația externalizată a aplicației

```
# Import configurație din fișier .env

spring.config.import=optional:file:./.env.properties,optional:file:./InternshipFinder/.env.properties

# Database Configuration (PostgreSQL)
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

# JWT Configuration
jwt.secret=${JWT_SECRET}
jwt.accessExpirationMs=${JWT_ACCESS_EXPIRATION_MS}
jwt.refreshExpirationMs=${JWT_REFRESH_EXPIRATION_MS}

# Mail Configuration
spring.mail.host=${SMTP_HOST:smtp.gmail.com}
spring.mail.port=${SMTP_PORT:587}
spring.mail.username=${SMTP_USERNAME}
spring.mail.password=${SMTP_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# Upload limits
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB

# Azure Storage Configuration
azure.storage.connection-string=${AZURE_STORAGE_CONNECTION_STRING:}
azure.storage.container-name=${AZURE_STORAGE_CONTAINER_NAME:}
azure.storage.sas.expiry-minutes=${AZURE_STORAGE_SAS_EXPIRY_MINUTES:}
```

Listarea A4.1. Fragment din fișierul application.properties

Anexa 5. Configurația securității Spring Security

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .cors(cors -> {})
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/api/auth/login",
                "/api/auth/logout",
                "/api/auth/refresh",
                "/api/auth/register",
                "/api/auth/request-otp",
                "/api/auth/verify-otp",
                "/api/auth/verify-email-otp",
                "/api/auth/resend-email-otp",
                "/api/auth/reset-password-otp"
            ).permitAll()
            .requestMatchers("/api/jobs/**").permitAll()
            .requestMatchers("/api/auth/me").authenticated()
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/settings/**").authenticated()
            .requestMatchers("/api/student/**").hasRole("STUDENT")
            .requestMatchers("/api/recruiter/**").hasRole("RECRUITER")
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .exceptionHandling(e -> e
            .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
        )
        .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Listarea A5.1. Fragment din SecurityConfig.java

Anexa 6. Componentele JWT, generare și filtrare

```
@Component
public class JwtTokenProvider {

    private final SecretKey key = Keys.secretKeyFor(SignatureAlgorithm.HS512);
    private final int expirationMs = 3600000; // 1 oră

    public String generateAccessToken(String email) {
        return Jwts.builder()
            .setSubject(email)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(key)
            .compact();
    }

    public String getEmailFromToken(String token) {
        return Jwts.parserBuilder().setSigningKey(key).build()
            .parseClaimsJws(token).getBody().getSubject();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token);
            return true;
        } catch (JwtException ex) {
            return false;
        }
    }
}
```

Listarea A6.1. Fragment din JwtTokenProvider.java

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain)
    throws ServletException, IOException {

    String token = null;
    if (request.getCookies() != null) {
        for (Cookie cookie : request.getCookies()) {
            if ("accessToken".equals(cookie.getName())) {
                token = cookie.getValue();
                break;
            }
        }
    }

    if (token != null && jwtProvider.validateToken(token)) {
        String email = jwtProvider.getEmailFromToken(token);
        var userDetails = userDetailsService.loadUserByUsername(email);

        var authToken = new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }

    filterChain.doFilter(request, response);
}
```

Listarea A6.2. Fragment din JwtAuthenticationFilter.java

Anexa 7. Controlerul de autentificare și serviciul de upload

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<?> login(@Valid @RequestBody LoginRequest request, HttpServletResponse
response) {
        AuthResponse authResponse = userService.login(request);
        AuthTokens tokens = authService.issueTokensForLogin(authResponse.getEmail());
        response.addHeader("Set-Cookie",
cookiesService.buildAccessCookie(tokens.getAccessJwt()).toString());
        response.addHeader("Set-Cookie",
cookiesService.buildRefreshCookie(tokens.getRefreshToken()).toString());

        return ResponseEntity.ok(authResponse);
    }

    @PostMapping("/refresh")
    public ResponseEntity<?> refresh(HttpServletRequest request, HttpServletResponse response) {
        String refreshRaw = cookiesService.readCookieValue(request, "refreshToken");
        String newAccessJwt = authService.refreshAccessToken(refreshRaw);
        response.addHeader("Set-Cookie",
cookiesService.buildAccessCookie(newAccessJwt).toString());
        return ResponseEntity.ok("REFRESHED");
    }

    @PostMapping("/request-otp")
    public ResponseEntity<String> requestOtp(@Valid @RequestBody OtpRequest req) {
        return ResponseEntity.ok(userService.sendOtpToEmail(req.getEmail()));
    }

    @PostMapping("/verify-otp")
    public ResponseEntity<String> verifyOtp(@Valid @RequestBody OtpVerificationRequest req) {
        userService.verifyOtp(req.getEmail(), req.getOtp());
        return ResponseEntity.ok("OTP_VERIFIED");
    }
}
```

Listarea A7.1. Fragment din AuthController.java

```
@Service
public class AzureBlobService implements StorageService {

    @Autowired
    private BlobContainerClient blobContainerClient;

    @Override
    public String uploadFile(MultipartFile file, String folder) {
        try {
            String blobName = folder + "/" + Instant.now().toEpochMilli() + "-" +
file.getOriginalFilename();
            BlobClient blobClient = blobContainerClient.getBlobClient(blobName);
            blobClient.upload(file.getInputStream(), file.getSize(), true);
            return blobClient.getBlobUrl();
        } catch (IOException e) {
            throw new RuntimeException("Upload failed", e);
        }
    }

    @Override
    public String toReadSasUrl(String storedUrlOrBlobName) {
        String blobName = extractBlobName(storedUrlOrBlobName);
        BlobClient blobClient = blobContainerClient.getBlobClient(blobName);
        BlobSasPermission permission = new BlobSasPermission().setReadPermission(true);
        var sasValues = new BlobServiceSasSignatureValues(OffsetDateTime.now().plusHours(1),
permission);

        return blobClient.getBlobUrl() + "?" + blobClient.generateSas(sasValues);
    }
}
```

Listarea A7.2. Fragment din FileUploadServiceImpl.java

Anexa 8. Componentele frontend pentru protecția rutelor

```
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

export default function ProtectedRoute({ children, role }) {
  const { auth } = useAuth();
  const location = useLocation();

  if (!auth) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (role) {
    const allowedRoles = Array.isArray(role) ? role : [role];
    if (!allowedRoles.includes(auth.role)) {
      return <Navigate to="/unauthorized" replace />;
    }
  }

  return children;
}
```

Listarea A8.1. Fișierul ProtectedRoute.jsx

```
const logout = useCallback(async () => {
  try {
    await axios.post('/auth/logout');
  } catch (err) {
    console.error('Logout error:', err);
  } finally {
    setAuth(null);
  }
}, []);

useEffect(() => {
  let refreshPromise = null;

  const interceptor = axios.interceptors.response.use(
    (response) => response,
    async (error) => {
      const originalRequest = error?.config;

      // Dacă eroarea nu este de autentificare sau cererea a fost deja reîncercată
      if (error?.response?.status !== 401 || originalRequest?._retry) {
        return Promise.reject(error);
      }

      originalRequest._retry = true;

      try {
        // Previne cererile multiple de refresh (reutilizează același Promise)
        if (!refreshPromise) {
          refreshPromise = axios.post('/auth/refresh').finally(() => {
            refreshPromise = null;
          });
        }
        await refreshPromise;

        // Reexecută cererea inițială eșuată
        return axios(originalRequest);
      } catch (refreshErr) {
        logout();
        return Promise.reject(refreshErr);
      }
    }
  );

  return () => axios.interceptors.response.eject(interceptor);
}, [logout]);
```

Listarea A8.2. Fragment din AuthContext.jsx (interceptor pentru refresh token)

Anexa 9. Modelul pentru token-uri OTP și transmiterea prin email

```
@Entity
@Table(name = "otp_tokens", indexes = {
    @Index(name = "idx_otp_user_purpose", columnList = "user_id,purpose"),
    @Index(name = "idx_otp_expires_at", columnList = "expires_at")
})
public class OtpToken {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private OtpPurpose purpose;

    @Column(name = "otp_code", nullable = false)
    private String otpCode;

    @Column(name = "target_email")
    private String targetEmail;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    @Column(name = "expires_at", nullable = false)
    private Instant expiresAt;

    @Column(name = "consumed_at")
    private Instant consumedAt;
}
```

Listarea A9.1. Fragment din OtpToken.java

```
@Override
public void sendOtpEmail(String toEmail, String otp) {
    if (smtpUsername == null || smtpUsername.isBlank()) {
        throw new IllegalStateException("SMTP username is missing. Set SMTP_USERNAME (or
spring.mail.username).");
    }

    if (smtpPassword == null || smtpPassword.isBlank()) {
        throw new IllegalStateException("SMTP password is missing. Set SMTP_PASSWORD (or
spring.mail.password). " +
            "For Gmail you must use an App Password.");
    }

    sendOtpEmailInternal(toEmail, otp, "Password reset request",
        "We received a request to reset the password for",
        brandName + " | Password Reset OTP");
}

private void sendOtpEmailInternal(String toEmail, String otp, String subject,
    String bodyPrefix, String emailTitle) {
    MimeMessage message = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(message, true, "UTF-8");

    helper.setFrom(mailFrom);
    helper.setTo(toEmail);
    helper.setSubject(subject);

    String htmlContent = buildOtpEmailTemplate(otp, emailTitle, bodyPrefix);
    helper.setText(htmlContent, true);

    mailSender.send(message);
}
```

Listarea A9.2. Fragment din metoda sendOtpEmail (EmailServiceImpl.java)

Anexa 10. Controlere pentru API pe roluri

```
@RestController
@RequestMapping("/api/student")
public class StudentController {
    @Autowired
    private StudentService studentService;
    @GetMapping("/profile")
    public ResponseEntity<StudentProfileDTO> getProfile() {
        return ResponseEntity.ok(studentService.getProfileForLoggedInUser());
    }

    @PutMapping("/profile")
    public ResponseEntity<String> updateProfile(@Valid @RequestBody StudentProfileDTO dto) {
        return ResponseEntity.ok(studentService.updateProfileForLoggedInUser(dto));
    }

    @PostMapping("/resume")
    public ResponseEntity<String> uploadResume(@RequestParam("file") MultipartFile file) throws
    Exception {
        return ResponseEntity.ok(studentService.uploadResumeForLoggedInUser(file));
    }
}
```

Listarea A10.1. Fragment din StudentController.java

```
@RestController
@RequestMapping("/api/recruiter")
public class RecruiterJobController {
    @Autowired
    private JobService jobService;
    @PostMapping("/jobs")
    public ResponseEntity<JobDetailsResponse> postJob(@RequestBody Job job, Principal principal)
    {
        Job saved = jobService.postJob(job, principal.getName());
        return ResponseEntity.ok(jobService.getRecruiterJobDetails(principal.getName(),
        saved.getId()));
    }

    @GetMapping("/jobs")
    public ResponseEntity<List<JobDetailsResponse>> getMyJobs(Principal principal) {
        return ResponseEntity.ok(jobService.getJobsByRecruiterAsDetails(principal.getName()));
    }
}
```

Listarea A10.2. Fragment din RecruiterJobController.java

```
@RestController
@RequestMapping("/api/admin")
public class AdminController {
    @Autowired
    private AdminService adminService;
    @GetMapping("/users")
    public ResponseEntity<List<User>> getAllUsers() {
        return ResponseEntity.ok(adminService.getAllUsers());
    }

    @PutMapping("/user/{id}/block")
    public ResponseEntity<String> toggleUserBlock(@PathVariable Long id) {
        adminService.toggleUserBlock(id);
        return ResponseEntity.ok("User block status updated");
    }

    @DeleteMapping("/user/{id}")
    public ResponseEntity<String> deleteUser(@PathVariable Long id) {
        adminService.deleteUser(id);
        return ResponseEntity.ok("User deleted successfully");
    }
}
```

Listarea A10.3. Fragment din AdminController.java

Anexa 11. Configurația interfeței utilizatorului, Vite, Axios și rutare

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    host: true,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        secure: false
      }
    }
  }
})
```

Listarea A11.1. Fișierul vite.config.js

```
import axios from "axios";

const axiosInstance = axios.create({
  baseURL: "/api",
  withCredentials: true,
});

export default axiosInstance;
```

Listarea A11.2. Fișierul axios.js

```
export default function App() {
  return (
    <>
      <Navbar />
      <Routes>
        { /* Rute Publice */ }
        <Route path="/" element={ <Home /> } />
        <Route path="/login" element={ <GuestRoute><Login /></GuestRoute> } />

        { /* Rute Protejate pe Roluri (Exemple) */ }
        <Route
          path="/student/dashboard"
          element={ <ProtectedRoute role="STUDENT"><StudentDashboard /></ProtectedRoute> }
        />
        <Route
          path="/recruiter/dashboard"
          element={ <ProtectedRoute role="RECRUITER"><RecruiterDashboard /></ProtectedRoute> }
        />
        <Route
          path="/admin/dashboard"
          element={ <ProtectedRoute role="ADMIN"><AdminDashboard /></ProtectedRoute> }
        />

        <Route path="*" element={ <NotFound /> } />
      </Routes>
    </>
  );
}
```

Listarea A11.3. Fragment din App.jsx (rutare protejată pe roluri)

```
import React, { createContext, useCallback, useContext, useEffect, useState } from 'react';
import axios from '../services/axios';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
```

```

const login = useCallback((userData) => {
  setUser({ email: userData.email, role: userData.role });
}, []);

const logout = useCallback(async () => {
  try {
    await axios.post('/auth/logout');
  } catch (err) {
    console.error(err);
  } finally {
    setUser(null);
  }
}, []);

// verifică sesiunea la deschiderea aplicației (Auto-login)
useEffect(() => {
  axios.get('/auth/me')
    .then(res => { if (res.data) login(res.data); })
    .catch(() => setUser(null))
    .finally(() => setLoading(false));
}, [login]);

if (loading) return null;

return (
  <AuthContext.Provider value={{ user, login, logout }}>
    {children}
  </AuthContext.Provider>
);
};
export const useAuthContext = () => useContext(AuthContext);

```

Listarea A11.4. Fragment din AuthContext.jsx (context, login, logout)

Anexa 12. Funcționalitatea CV Builder

```
export default function CvBuilder() {
  const { t } = useTranslation();
  const [enabledSections] = useState({ experience: true, projects: true });

  const schema = useMemo(() => createCvBuildersSchema(t), [t]);

  const { register, control, getValues, trigger } = useForm({
    defaultValues: {
      personal: { fullName: '', email: '' },
      summary: '',
      education: [{ school: '', degree: '', start: '', end: '' }],
      experience: [{ company: '', role: '', start: '', end: '' }]
    },
    resolver: yupResolver(schema, { context: { enabledSections } })
  });

  const educationArray = useFieldArray({ control, name: 'education' });
  const experienceArray = useFieldArray({ control, name: 'experience' });
}
```

Listarea A12.1. Fragment din CvBuilder.jsx (inițializare formular și validare)

```
const ensurePdfUnicodeFonts = async (pdf) => {
  try {
    if (!pdfFontsReadyRef.current) {
      const res = await fetch('/fonts/NotoSans-Regular.ttf');
      if (!res.ok) return false;

      const buffer = await res.arrayBuffer();
      // Conversie rapidă în Base64 utilizând standardul btoa direct pe bytes-uri textari
      const base64Font = btoa(String.fromCharCode(...new Uint8Array(buffer)));

      pdfFontDataRef.current = { regular: base64Font };
      pdfFontsReadyRef.current = true;
    }

    pdf.addFileToVFS('NotoSans-Regular.ttf', pdfFontDataRef.current.regular);
    pdf.addFont('NotoSans-Regular.ttf', 'NotoSans', 'normal');
    return true;
  } catch {
    return false;
  }
};
```

Listarea A12.2. Fragment din CvBuilder.jsx (încărcare fonturi Unicode pentru PDF)

```
const buildPdf = async (data) => {
  const pdf = new jsPDF({ orientation: 'p', unit: 'mm', format: 'a4' });
  await ensurePdfUnicodeFonts(pdf);

  // Desenează Header-ul documentului PDF
  pdf.setFillColor(79, 70, 229);
  pdf.rect(0, 0, pdf.internal.pageSize.getWidth(), 22, 'F');
  pdf.setTextColor(255);
  pdf.text(data.personal.fullName || 'CV', 14, 12);

  return pdf;
};

const exportPdf = async () => {
  if (!(await trigger())) return;
  const pdf = await buildPdf(getValues());
  pdf.save('cv.pdf');
};
```

Listarea A12.3. Fragment din CvBuilder.jsx (generare și export PDF)

```
const saveCv = async () => {
  try {
    if (!(await trigger())) return;
  }
}
```

```

const pdf = await buildPdf(getValues());
const blob = pdf.output('blob');
const file = new File([blob], 'cv.pdf', { type: 'application/pdf' });

const formData = new FormData();
formData.append('file', file);

await axios.post('/student/resume', formData);
showToast('cv-save', 'success', t('cvBuilder.toasts.saved'));
} catch (err) {
showToast('cv-save', 'error', t('cvBuilder.toasts.saveFailed'));
}
};

```

Listarea A12.4. Fragment din CvBuilder.jsx (salvare CV ca resume)

```

import * as yup from 'yup';

export const createCvBuildersSchema = (t) => {
  const itemSchema = yup.object({
    schoolOrCompany: yup.string().trim().required(t('validation.required')),
    start: yup.string().trim().required().matches(/^d{4}$/),
    end: yup.string().trim().required().matches(/^d{4}$/)
  });

  return yup.object({
    personal: yup.object({
      fullName: yup.string().trim().required(),
      email: yup.string().trim().email().required()
    }),
    summary: yup.string().trim().required(),
    education: yup.array().of(itemSchema).min(1),

    // validare condiționată în funcție de secțiunile active din context
    experience: yup.array().when('$enabledSections', {
      is: (env) => Boolean(env?.experience),
      then: (schema) => schema.of(itemSchema).min(1),
      otherwise: (schema) => schema.notRequired()
    }),

    projects: yup.string().trim().test('projects-enabled', function (v) {
      if (!this?.options?.context?.enabledSections?.projects) return true;
      return String(v || '').trim().length >= 10;
    })
  });
};

```

Listarea A12.5. Fragment din cvBuilderSchema.js (validare condiționată)

DECLARAȚIE PRIVIND ASUMAREA RĂSPUNDERII

Subsemnatul STRATON ALEXANDRU, absolvent al Universității de Stat din Moldova, programul de Licență 0613.5 INFORMATICĂ APLICATĂ declar pe propria răspundere că teza de licență cu tema „PROIECTAREA ȘI DEZVOLTAREA UNUI PORTAL WEB PENTRU GESTIONAREA INTERNSHIP-URILOR STUDENȚILOR este rezultatul muncii mele, este realizată pe baza propriilor cercetări și pe baza informațiilor obținute din surse care au fost citate și indicate, conform normelor etice.

De asemenea, declar că sursele utilizate în teză, inclusiv cele din Internet, precum și fragmentele generate de aplicații deschise de inteligență artificială generativă, sunt indicate cu respectarea regulilor de evitare a plagiatului:

1. Fragmentele de text reproduse sunt scrise între ghilimele, cu referința precisă a sursei;
2. Rezumarea ideilor altor autori și/sau redarea/reformularea lor în cuvinte proprii conține referința precisă a sursei.
3. Fragmentele generate de Open AI sunt menționate explicit.

Conștientizez că, în caz contrar, urmează să suport consecințele în conformitate cu legislația în vigoare.

Nume, prenume: STRATON ALEXANDRU

Semnătura: _____

Data: 03.06.2026

DECLARAȚIE PRORTO IVIND CONTRIBUȚIA PERSONALĂ ȘI, DUPĂ CAZ, UTILIZAREA INSTRUMENTELOR DE INTELIGENȚĂ ARTIFICIALĂ

Subsemnatul(a) STRATON ALEXANDRU, autor / autoare a tezei / proiectului cu titlul „PROIECTAREA ȘI DEZVOLTAREA UNUI PORTAL WEB PENTRU GESTIONAREA INTERNSHIP-URILOR STUDENȚILOR”, declar pe propria răspundere că teza/proiectul/lucrarea a fost elaborată cu respectarea normelor de integritate academică și că reprezintă rezultatul contribuției mele personale, cu excepția elementelor citate și indicate corespunzător.

De asemenea, declar următoarele cu privire la utilizarea instrumentelor de inteligență artificială:

- ☐ Nu am utilizat instrumente de inteligență artificială la elaborarea lucrării.
- ☐ Am utilizat instrumente de inteligență artificială exclusiv pentru suport tehnic (corectare lingvistică, formatare, organizare preliminară).
- ☒ Am utilizat instrumente de inteligență artificială într-o manieră asistată și declar mai jos natura utilizării, fără a afecta caracterul personal al contribuției intelectuale.

Descrierea utilizării declarate, dacă este cazul:

Data: 03.06.2026

Semnătura autorului / autoarei: _____

